Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Multi-Sensor Spatial Intelligence and Analytics Platform (MSSIA)

Project Engineering

Year 4

Nokuvimba Natalie Chiyaka

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

__Nokuvimba Natalie Chiyaka_____

Acknowledgements

Use this section to acknowledge anyone, if you wish to, who might have helped during your project.

- Ben Kinsella

Table of Contents

1 Summary 8
2 Poster 9
3 Introduction 10
4 Background 12
4.1 Indoor Environmental Monitoring 12
4.2 Wi-Fi Signal Strength as a Spatial Metric 12
4.3 IoT Architectures for Building Monitoring 13
4.4 Related Work and Commercial Alternatives 14
5 Project Architecture 16
6 Project Plan 17
6.1 Work Breakdown 17
6.2 Sprint Structure 17
6.3 Timeline 18
6.4 Lessons Learned 20
7 Hardware 20
7.1 ESP32 Microcontroller 21
7.2 DHT22 Temperature & Humidity Sensor 22
7.3 MQ-135 Air Quality Sensor 23
7.4 Wiring and Circuit Design 26
7.5 Power Supply 26
7.6 Problems Encountered 26
8 Software : Backend 27
8.1 FastAPI Framework 27

8.2 Database Design (PostgreSQL) and the SQLite to PostgreSQL Migration 28
8.3 API Endpoints 29
8.4 JWT Authentication 30
8.5 Data Ingestion and Validation 31
8.6 Real-Time Updates (Server-Sent Events) 32
8.7 Environmental Alerts 33
8.8 Problems Encountered 33
9 Software: Frontend 35
9.1 Next.js Architecture 35
9.2 Admin Studio 35
9.3 User Viewer 36
9.4 Heatmap Rendering 37
9.5 Floor Plan Interaction (Zoom, Pan, Pin Placement) 37
9.6 Data Export (CSV and PowerPoint) 38
9.7 Problems Encountered 38
10 Cloud Deployment 39
10.1 Infrastructure Overview 40
10.2 AWS EC2 Setup 41
10.3 FastAPI as a system Service 41
10.4 Nginx Reverse Proxy and HTTPS 42
10.5 DuckDNS Domain 43
10.6 Vercel Frontend Hosting 43
10.7 DNS Chain and Request Flow 44
10.8 Problems Encountered 44

11 Testing and Evaluation 46
11.1 Backend Unit Tests (pytest) 46
11.2 Frontend Unit Tests (Jest) 48
11.2.1 Problems Encountered 49
11.3 End-to-End Testing — Playwright 49
11.3.1 Problems Encountered 51
11.4 Hardware Testing 52
11.5 Results 53
12 Development Tools and Process 53
12.1 Version Control: Git and GitHub 53
12.2 Project Management: Jira 54
12.3 UI Design : Figma 55
12.4 Documentation : OneNote 56
12.5 AI-Assisted Development (Claude, Amazon Q, ChatGpt) 56
12.6 Peer Collaboration 57
13 Ethics 57
13.1 Data Privacy 57
13.2 Security 58
13.3 Consent and Transparency 58
13.4 Responsible Use of AI Tools 58
13.5 Environmental Impact 59
13.6 Accessibility 59
14 Limitations and Future Work 60
14.1 MQ-135 Calibration 60

14.2 ESP32 Communication Security 60

14.3 Single Administrator Account 61

14.4 No User Authentication on the Public Viewer 61

14.5 ESP32 Warm-Up Period 61

14.6 Heatmap Colour Accessibility 62

14.7 Scalability 62

14.8 Future Feature Opportunities 62

15 Conclusion 62

16 References 64

16.1 Appendix A Bill of Materials 68

16.2 Appendix B – Project Resources and Links 69

1 Summary

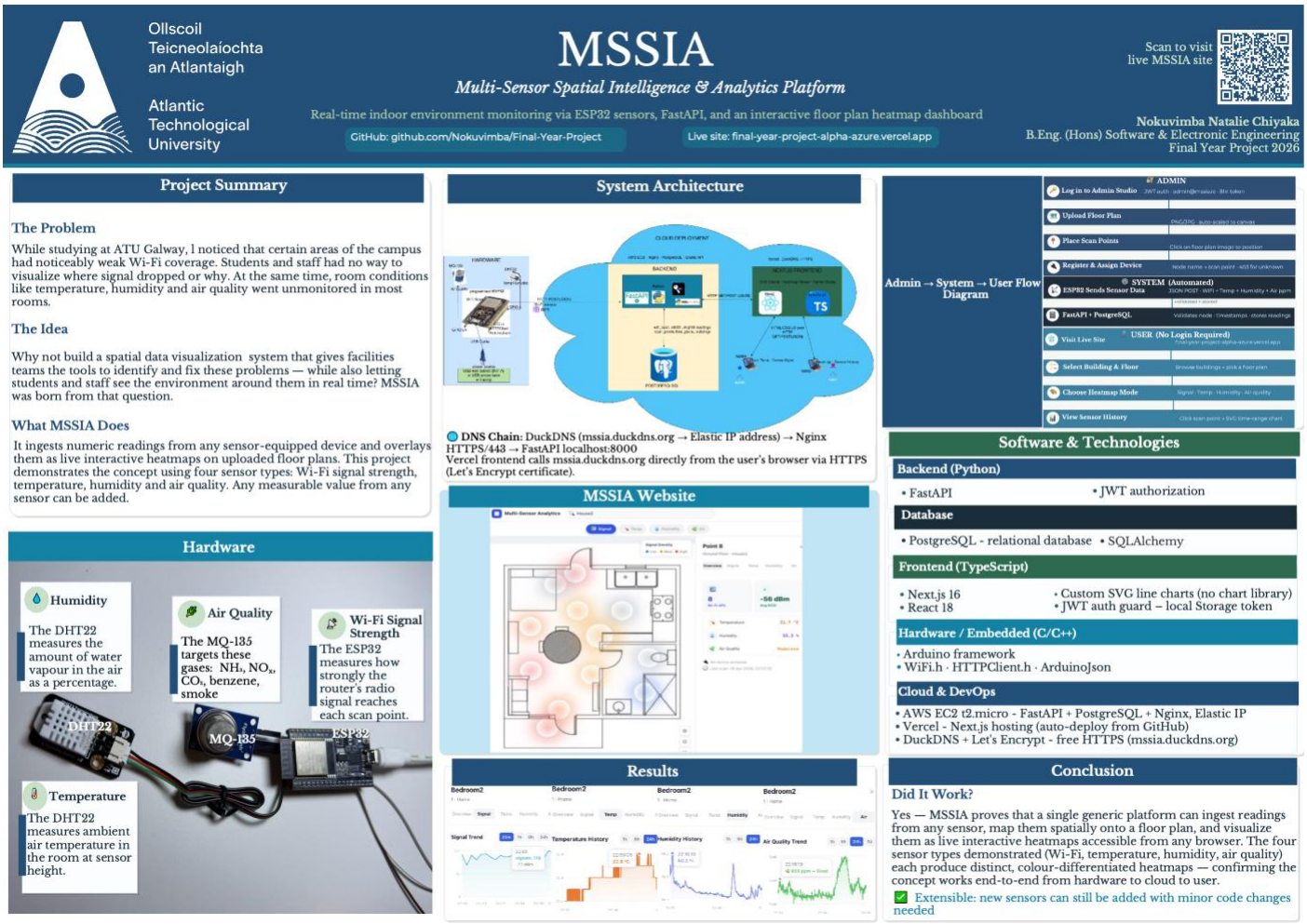
While studying at ATU Galway, I noticed that certain areas of the campus had noticeably weak Wi-Fi coverage. Students and staff had no way to visualise where signal dropped or why, and room conditions such as temperature, humidity, and air quality went unmonitored in most rooms. This led to the idea behind this project: a spatial data visualisation system that would give facilities teams the tools to identify these problems, while also letting students and staff see the environment around them in real time.

The goal of this project was to build a generic Internet of Things platform that ingests numeric readings from any sensor-equipped device and overlays them as live, interactive heatmaps on uploaded floor plans. The scope covered the full stack from hardware to cloud to user interface, demonstrating the concept with four sensor types: Wi-Fi signal strength, temperature, humidity, and air quality.

The approach was a three-layer architecture. An ESP32 microcontroller with a DHT22 and an MQ-135 sensor collects readings and transmits them via HTTP POST. A FastAPI backend validates, timestamps, and stores the data in a PostgreSQL database hosted on AWS EC2. A Next.js frontend deployed on Vercel presents the data through an authenticated admin studio and a public user view. Key features include four interactive heatmap modes, JWT authentication, Server-Sent Events for real-time updates, and automated unit tests. The main technologies used are ESP32 with C++, FastAPI (Python), PostgreSQL, Next.js (TypeScript), Nginx, and Let's Encrypt HTTPS via DuckDNS.

What was accomplished is a fully deployed system accessible at final-year-project-alpha-azure.vercel.app, with the API served from mssia.duckdns.org. The main conclusion is that MSSIA proves a single generic platform can ingest readings from any sensor, map them spatially onto a floor plan, and visualise them as live interactive heatmaps accessible from any browser.

2 Poster


Ollscoil Teicneolaíochta an Atlantaigh
Atlantic Technological University
MSSIA
Multi-Sensor Spatial Intelligence & Analytics Platform
Real-time indoor environment monitoring via ESP32 sensors, FastAPI, and an interactive floor plan heatmap dashboard
GitHub: github.com/Nokuvimba/Final-Year-Project
Live site: final-year-project-alpha-azure.vercel.app
Scan to visit live MSSIA site
Nokuvimba Natalie Chiyaka
B.Eng. (Hons) Software & Electronic Engineering
Final Year Project 2026
Project Summary
The Problem
While studying at ATU Galway, I noticed that certain areas of the campus had noticeably weak Wi-Fi coverage. Students and staff had no way to visualize where signal dropped or why. At the same time, room conditions like temperature, humidity and air quality went unmonitored in most rooms.
The Idea
Why not build a spatial data visualization system that gives facilities teams the tools to identify and fix these problems — while also letting students and staff see the environment around them in real time? MSSIA was born from that question.
What MSSIA Does
It ingests numeric readings from any sensor-equipped device and overlays them as live interactive heatmaps on uploaded floor plans. This project demonstrates the concept using four sensor types: Wi-Fi signal strength, temperature, humidity and air quality. Any measurable value from any sensor can be added.
Hardware
Humidity
The DHT22 measures the amount of water vapour in the air as a percentage.
Air Quality
The MQ-135 targets these gases: NH₃, NOₓ, CO₂, benzene, smoke
Wi-Fi Signal Strength
The ESP32 measures how strongly the router's radio signal reaches each scan point.
DHT22
MQ-135
ESP32
Temperature
The DHT22 measures ambient air temperature in the room at sensor height.
System Architecture
DNS Chain: DuckDNS (mssia.duckdns.org → Elastic IP address) → Nginx HTTPS/443 → FastAPI localhost:8000
Vercel frontend calls mssia.duckdns.org directly from the user's browser via HTTPS (Let's Encrypt certificate).
MSSIA Website
Results
Admin → System → User Flow Diagram
ADMIN
Log in to Admin Studio
Upload Floor Plan
Place Scan Points
Register & Assign Device
SYSTEM (Automated)
ESP32 Sends Sensor Data
FastAPI + PostgreSQL
USER (No Login Required)
Visit Live Site
Select Building & Floor
Choose Heatmap Mode
View Sensor History
Software & Technologies
Backend (Python)
• FastAPI
• JWT authorization
Database
• PostgreSQL - relational database
• SQLAlchemy
Frontend (TypeScript)
• Next.js 16
• React 18
• Custom SVG line charts (no chart library)
• JWT auth guard – local Storage token
Hardware / Embedded (C/C++)
• Arduino framework
• WiFi.h · HTTPClient.h · ArduinoJson
Cloud & DevOps
• AWS EC2 t2.micro - FastAPI + PostgreSQL + Nginx, Elastic IP
• Vercel - Next.js hosting (auto-deploy from GitHub)
• DuckDNS + Let's Encrypt - free HTTPS (mssia.duckdns.org)
Conclusion
Did It Work?
Yes — MSSIA proves that a single generic platform can ingest readings from any sensor, map them spatially onto a floor plan, and visualize them as live interactive heatmaps accessible from any browser. The four sensor types demonstrated (Wi-Fi, temperature, humidity, air quality) each produce distinct, colour-differentiated heatmaps — confirming the concept works end-to-end from hardware to cloud to user.
Extensible: new sensors can still be added with minor code changes needed

3 Introduction

Modern buildings are full of invisible data. Wi-Fi signals fluctuate room by room, air quality varies with occupancy and ventilation, and temperature and humidity differ across floors and zones, yet most buildings provide no way to see any of this information at a glance. Facilities teams often only learn about poor Wi-Fi coverage or stuffy rooms after complaints are raised, and by then the underlying issue may have gone unaddressed for some time. The people who occupy these spaces every day, whether they are employees in an office building, patients in a hospital, or students and staff on a university campus, have no insight at all into the conditions of the rooms they use.

The goal of this project was to build a generic Internet of Things platform that addresses this problem by collecting numeric sensor readings from low-cost microcontrollers placed throughout a building and visualising them as live, interactive heatmaps overlaid on uploaded floor plans. The system does not attempt to fix environmental issues automatically; rather, it is a diagnostic and awareness tool that makes invisible building data visible so that informed decisions can be made. It was designed to be generic so that any sensor producing a numeric value, not just the four demonstrated in this project, can be added with minimal changes to the underlying architecture.

The motivation for the project came from my own experience as a student at ATU Galway, where I noticed that certain areas of the campus had noticeably weak Wi-Fi coverage, and that room conditions such as temperature, humidity, and air quality were not monitored or visible anywhere. This observation was the starting point, but the system itself was designed from the beginning to generalise beyond a single campus. A library, an office block, a care home, and a school all have the same underlying need: a way to see environmental and signal conditions across their spaces in real time. MSSIA was designed to serve any of them.

The scope of this project covers the full stack required to deliver such a system. On the hardware side, this includes an ESP32 microcontroller paired with a DHT22 sensor for temperature and humidity, and an MQ-135 sensor for air quality, together with Wi-Fi scanning to capture signal strength. On the software side, this includes a Python backend built with

FastAPI, a PostgreSQL database for persistent storage, and a TypeScript frontend built with Next.js. On the deployment side, this includes hosting the backend on an Amazon Web Services EC2 server secured with HTTPS, and hosting the frontend on Vercel with automatic deployments from GitHub. The project also includes automated testing for both layers, real-time updates using Server-Sent Events, JWT-based authentication for administrators, and data export in CSV and PowerPoint formats.

The remainder of this report is organised as follows. Section 4 introduces relevant background on indoor environmental monitoring and the technical concepts behind Wi-Fi signal strength as a spatial metric. Section 5 presents the overall project architecture. Section 6 outlines the project plan and how it was managed across the academic year. Sections 7 through 9 describe the hardware, backend software, and frontend software respectively, including the main problems encountered and how they were resolved in each layer. Section 10 details the cloud deployment process. Section 11 presents the testing and evaluation work carried out on the system. Section 12 describes the development tools and process, including the use of AI-assisted development. The report closes with sections on ethics, limitations and future work, and a conclusion summarising what was achieved.

4 Background

This section introduces the technical and contextual background required to understand the design and purpose of MSSIA. It is organized into four subsections: the importance of indoor environmental monitoring, the technical meaning of Wi-Fi signal strength as a spatial metric, common Internet of Things architectures for building monitoring, and a review of related work and commercial alternatives.

4.1 Indoor Environmental Monitoring

Indoor environmental quality has a direct impact on the comfort, productivity, and health of building occupants. The three metrics most commonly used to characterise it are temperature, relative humidity, and air quality. Temperature affects both perceived comfort and cognitive performance; research has shown that productivity falls measurably in rooms that are too hot or too cold [1]. Relative humidity, which is the amount of water vapour in the air expressed as a percentage of the maximum the air can hold at a given temperature, affects both comfort and the spread of airborne viruses, with the optimal range for human occupancy generally considered to be between 40% and 60% [2]. Air quality, measured through the concentration of pollutants such as carbon dioxide, nitrogen oxides, and volatile organic compounds, affects respiratory health, concentration, and long-term wellbeing [3].

In most buildings today, these three metrics are either unmonitored entirely or monitored only by commercial Building Management Systems (BMS) that are expensive, closed-source, and installed primarily in large commercial premises such as hospitals and corporate offices [4]. Smaller buildings, educational institutions, care homes, and older facilities typically have very little or no environmental monitoring at all. When problems arise, rooms that are too warm, poorly ventilated, or too dry are usually only identified through occupant complaints, by which point the conditions have already affected the people inside.

4.2 Wi-Fi Signal Strength as a Spatial Metric

Alongside environmental conditions, Wi-Fi signal strength is an increasingly important dimension of indoor building quality. Weak or inconsistent wireless coverage affects productivity, the ability of occupants to use cloud-based services, and in some cases safety-

critical systems that rely on wireless connectivity. Wi-Fi signal strength is measured using a metric called the Received Signal Strength Indicator (RSSI), expressed in decibels relative to one milliwatt (dBm) [5]. Because RSSI is expressed on a logarithmic scale, the values are always negative: a value closer to zero represents a stronger signal, while a more negative value represents a weaker one. Values of -50 dBm or above are generally considered excellent, while values below -70 dBm are considered weak and may cause intermittent connectivity issues.

RSSI is not a direct measurement of signal quality in the way that throughput is, but it serves as a reliable indicator of coverage because it reflects the amount of radio frequency energy reaching the receiver from a given access point. Signal strength inside a building is affected by the distance to the access point, the materials of the walls, ceilings, and furniture between the transmitter and receiver, and interference from other wireless devices operating in the same frequency bands. Because the indoor radio environment is complex and difficult to model analytically, directly sampling RSSI at different points within a building provides a practical and accurate way to map coverage.

Mapping RSSI values spatially onto a floor plan is known as Wi-Fi heatmapping. It is widely used by network engineers as a diagnostic tool, but most heatmapping software requires specialised equipment or manual site surveys [6]. MSSIA takes the same concept and makes it continuous, automated, and visible to both facilities teams and end users by using low-cost microcontrollers as permanent monitoring nodes.

4.3 IoT Architectures for Building Monitoring

An Internet of Things (IoT) system is typically described as a three-layer architecture consisting of a perception layer, a network layer, and an application layer [10]. The perception layer is where physical sensors collect data from the environment. The network layer is where that data is transmitted to a central system, usually over the internet. The application layer is where the data is stored, processed, and presented to a user, typically through a web or mobile interface. MSSIA follows this pattern directly: the ESP32 with its attached sensors constitutes the perception layer, the FastAPI backend and database constitute the network and storage layer, and the Next.js frontend constitutes the application layer.

Within this broad architecture, there were several choices to be made about the specific protocols and technologies to be used. The two most common protocols for device-to-server communication in IoT systems are HTTP and MQTT. HTTP is a request-response protocol that is widely supported, easy to debug, and integrates naturally with REST APIs, but it introduces a small overhead per message because each request opens a new connection [10]. MQTT, by contrast, is a publish-subscribe protocol designed specifically for low-power devices and high-volume message passing, but it requires a dedicated broker such as Mosquitto or HiveMQ to sit between devices and the server[40]. For a system that sends sensor readings every thirty seconds rather than many times per second, and that only has a handful of devices, HTTP is both simpler and sufficient. It was therefore the protocol chosen for MSSIA.

The choice of microcontroller is also significant. For low-cost indoor monitoring, the ESP32 has become the standard choice in hobbyist and small-scale commercial IoT work, largely because it integrates Wi-Fi, Bluetooth, and a dual-core processor at a very low price point [11]. It is programmed using the same Arduino framework familiar to most embedded engineers, and supports the HTTP client libraries needed to post JSON payloads to a cloud backend.

4.4 Related Work and Commercial Alternatives

Several commercial products already address aspects of the problem MSSIA solves, but none combine them in the way this project does. Ekahau and NetSpot are two of the most widely used Wi-Fi site survey tools [6]. Both produce detailed heatmaps of wireless coverage, but they are designed for one-off surveys rather than continuous monitoring, require a trained engineer to walk the building with a laptop, and focus exclusively on Wi-Fi rather than environmental metrics. At the other end of the scale, building management systems such as Siemens Desigo and Honeywell Enterprise Buildings Integrator monitor environmental conditions at scale, but they are proprietary, expensive, and aimed at large commercial premises [7]. Consumer-grade environmental monitors such as Airthings and Netatmo track air quality and humidity in a single room but provide no spatial visualisation across an entire building.

Academic research in the same area has explored indoor Wi-Fi fingerprinting for positioning and occupancy detection [8], dense sensor networks for environmental monitoring [9], and cloud-based IoT platforms for aggregating sensor data [10]. However, most published systems focus on one metric at a time, require specialised hardware, or stop at the data-collection stage without producing a user-facing visualisation.

MSSIA occupies a distinctive middle ground. It is continuous rather than survey-based, multi-metric rather than single-metric, low-cost rather than enterprise-grade, and explicitly designed from the beginning to be generic. This means that any numeric sensor can be added to the system without modifying the core architecture. The four sensor types demonstrated in this project (Wi-Fi signal strength, temperature, humidity, and air quality) are chosen to prove the concept, not to exhaust the system's capabilities.

5 Project Architecture

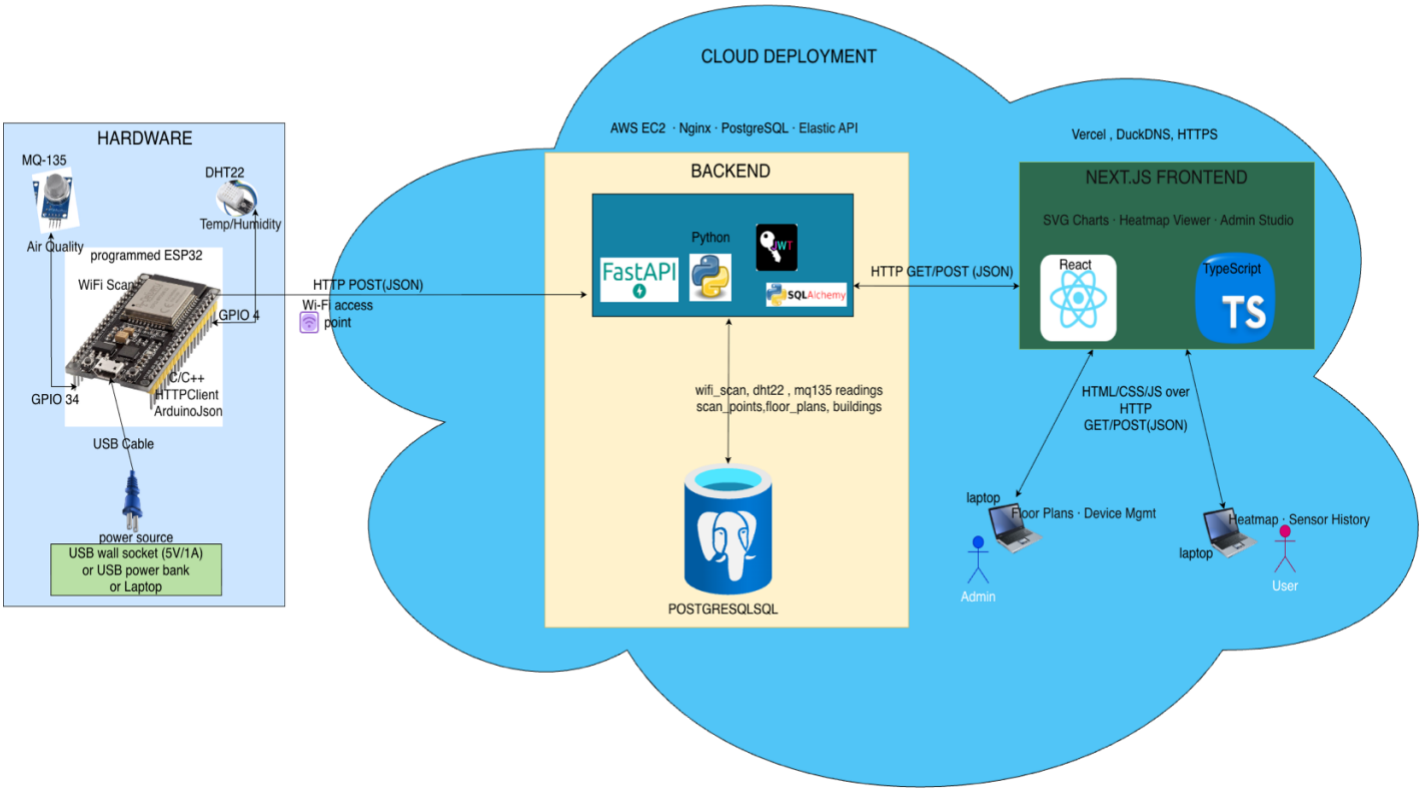


Figure 5.1 Architecture Diagram

6 Project Plan

The project was planned and managed using Jira, following an Agile methodology organized around Scrum sprints. I chose this approach because the project involved many distinct technical areas: hardware, backend, frontend, cloud deployment, and testing. Scrum's iterative structure allowed each area to be addressed in focused increments while keeping the overall direction clear.

6.1 Work Breakdown

At the start of the academic year, I broke the project down into seven high-level epics, each representing a major functional area of the system: Hardware Setup, Backend API and Database Design, Frontend UI, Machine Learning and Analytics, System Testing and Evaluation, and Documentation. Each epic was then decomposed into smaller user stories and tasks with clear acceptance criteria, allowing progress to be tracked at a granular level while keeping sight of the overall project goals.

Tasks were written from a user or system perspective rather than as pure implementation to-dos. For example, rather than a task stating "write a function to parse the JSON payload", the equivalent Scrum story read "the system should accept and store sensor readings from the ESP32 via HTTP POST". This framing kept the focus on what the system needed to do rather than on the implementation detail alone.

6.2 Sprint Structure

Sprints were not fixed to a standard two-week cycle. Instead, their duration was adjusted according to the scope and complexity of the work planned for each sprint. Simpler sprints lasted one to two weeks, while sprints involving more difficult integration work, such as connecting multiple sensors to the ESP32 or building the first version of the heatmap viewer — ran for three weeks or longer. I found this flexible approach more practical than fixed-length sprints for a solo project where academic workload also varied across the year.

Each sprint began with a planning session in which I selected issues from the backlog that made up that sprint's scope. At the end of each sprint, a retrospective was held and documented in

Confluence, reflecting on what had gone well, what could be improved, and what should be carried forward. These retrospectives were particularly useful for recognising recurring patterns early. For example, I noticed that sprints were not always being closed cleanly before new work started, causing tasks from different sprints to blur together. Subsequent sprints became more disciplined as a result of identifying this pattern.

6.3 Timeline

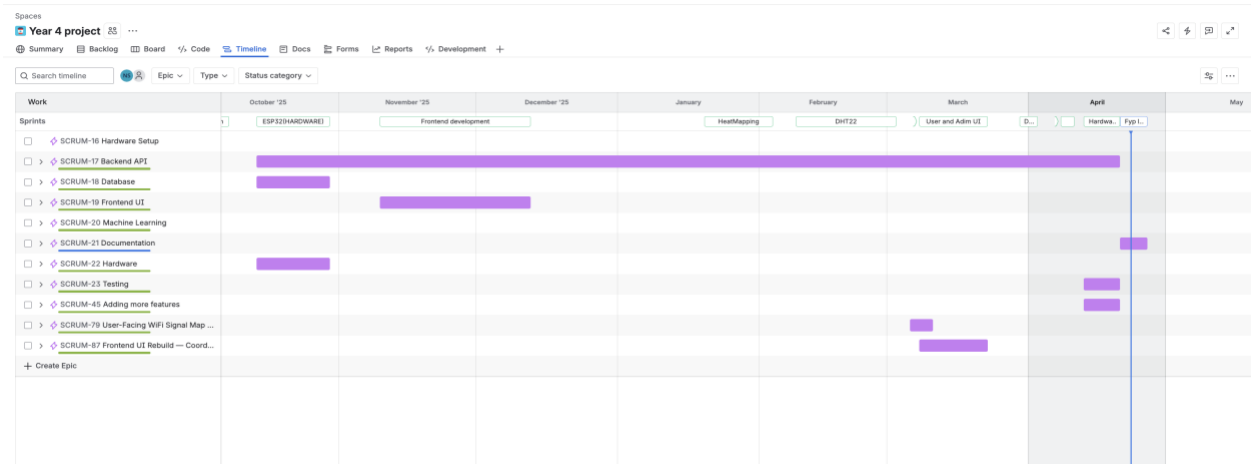
The high-level timeline of the project is summarised in Table 6.1. The earliest sprints focused on research, establishing the development environment, and getting a basic Wi-Fi scan working on the ESP32. The middle period of the academic year was dominated by backend and frontend development in parallel, with the two layers progressively integrated. The final months focused on cloud deployment, testing, feature refinement, and documentation.

Table 6.1 Project Timeline

Phase	Period	Key Deliverables
Proposal and research	Late September – Early October 2025	Project idea approved, supervisor assigned, tools set up
ESP32 Wi-Fi scan and JSON payload	October – Early November 2025	Working Wi-Fi scanner sending HTTP POST to backend
Backend development and SQLite-to-PostgreSQL migration	October – November 2025	FastAPI backend receiving and storing sensor data
Building and floor plan data model	Mid-November 2025	Building management API and CRUD endpoints

Frontend setup and heatmap visualization	November – December 2025	Next.js admin studio with first heatmap view
DHT22 sensor integration	January – February 2026	Temperature and humidity heatmap modes
MQ-135 sensor integration	February – March 2026	Air quality heatmap mode
JWT authentication	March 2026	Secure admin login and protected routes
Cloud deployment (AWS EC2 + Vercel)	March – April 2026	Live HTTPS deployment via DuckDNS
Feature refinement: zoom/pan, export, SSE, alerts	April 2026	Full feature set complete
System testing, documentation and report	April 2026	60 automated tests, logs, poster and report

Figure 6.1 Screenshot of the Jira Timeline



6.4 Lessons Learned

Running a project across a full academic year produced several practical lessons about planning that shaped how the project was managed over time.

The initial set of epics created at the start of the year included areas such as Machine Learning and Analytics, which were eventually dropped as the project direction became clearer. What began as a broad plan evolved significantly as technical decisions were made and new requirements emerged. New sprints and tasks were added throughout the year that had not been anticipated at the outset. This was a normal part of the process and reflected the reality that software projects rarely follow their initial plan exactly — what matters is that the plan is adapted rather than abandoned.

Sprint management also improved considerably over the course of the project. In the early stages, I tended to over-assign tasks to each sprint, which meant that sprints regularly ran past their planned end date without being formally closed. This made it harder to track what had genuinely been completed and what was still in progress. Over time, I found that it was more effective to under-assign tasks at the start of each sprint and then pull additional work from the backlog once the planned tasks were done. This approach kept sprints shorter, cleaner, and easier to review at retrospective time.

Finally, I found that regular in-person meetings with my supervisor were consistently the most valuable planning input throughout the project and helped maintain direction during periods of technical uncertainty.

7 Hardware

The hardware layer of MSSIA consists of a single ESP32 microcontroller mounted on a breadboard with two sensors attached: a DHT22 for temperature and humidity, and an MQ-135 for air quality. The ESP32's built-in Wi-Fi radio provides the fourth data source by scanning for nearby wireless access points and recording their signal strength. This section describes each component in detail, including its function, technical specifications from the datasheet, how it is wired into the circuit, and the relevant code used to read from it.

7.1 ESP32 Microcontroller

The ESP32 is a low-cost, low-power system-on-chip microcontroller developed by Espressif Systems [11]. It integrates a dual-core Xtensa LX6 processor running at up to 240 MHz, 520 KB of SRAM, 4 MB of flash storage, and built-in Wi-Fi (802.11 b/g/n) and Bluetooth radio transceivers, making it well suited to Internet of Things applications that require both local sensing and wireless connectivity. The ESP32 is programmed using the Arduino framework via the Arduino IDE or PlatformIO and supports C/C++ with a rich ecosystem of libraries for peripheral communication, HTTP networking, and JSON serialization [12].

For this project, the ESP32 performs three roles simultaneously: it scans nearby Wi-Fi access points using the built-in Wi-Fi radio, reads sensor data from the DHT22 and MQ-135 over GPIO pins, and transmits the combined data to the FastAPI backend via an HTTP POST request. The sketch runs in a continuous loop, performing a scan-and-send cycle approximately every 30 seconds. The device is programmed over USB and once deployed, powered by a standard USB wall adapter, eliminating the need for a laptop during normal operation.

The following code shows the core structure of the scanning loop:

```
// ========================================================
void startWiFiScan() {
  if (WiFi.status() != WL_CONNECTED) return; // only scan if connected
  Serial.println("🔍 Starting async Wi-Fi scan...");
  WiFi.scanDelete();              // clear old results
  WiFi.scanNetworks(true, true);  // async mode, include hidden networks
}
```

```
void postScannedNetworks(int16_t networksFound) {
  if (networksFound <= 0) return;

  StaticJsonDocument<4096> doc;

  // Top-level fields
  doc["node"] = NODE_TAG;

  // Scans array — one object per SSID found
  JsonArray scans = doc.createNestedArray("scans");

  for (int i = 0; i < networksFound; ++i) {
    JsonObject o = scans.createNestedObject();
    o["ssid"]    = WiFi.SSID(i);
    o["bssid"]   = WiFi.BSSIDstr(i);
    o["rssi"]    = WiFi.RSSI(i);
    o["channel"] = WiFi.channel(i);
    o["enc"]     = WiFi.encryptionType(i);
    // NOTE: "ts" (millis) intentionally removed — server adds real UTC timestamp
  }
```

The WiFi.scanNetworks() function triggers a full scan of all available access points and returns the number found. Each access point's SSID, BSSID (hardware address), RSSI, channel, and encryption type are then added to a JSON array using the ArduinoJson library [13]. This payload is later extended with DHT22 and MQ-135 readings before being transmitted.

7.2 DHT22 Temperature & Humidity Sensor

The DHT22 is a digital temperature and humidity sensor manufactured by Aosong Electronics [14]. It uses a capacitive humidity sensing element and a thermistor to measure ambient conditions, outputting a calibrated digital signal over a single-wire serial protocol. Table 7.1 summarizes the key specifications from the datasheet.

Table 7.1 DHT22 Specifications

Parameter	**Value**
Temperature range	−40°C to +80°C
Temperature accuracy	±0.5°C
Humidity range	0–100% RH
Humidity accuracy	±2–5% RH
Sampling rate	0.5 Hz (one reading every 2 seconds)
Operating voltage	3.3V – 5V
Communication	Digital one-wire protocol

The DHT22 is connected to the ESP32 on GPIO 4, with VCC connected to the 3.3V rail and GND to the ESP32's ground pin. The DHT22 module used in this project is a pre-assembled 3-pin version with the pull-up resistor built onto the module board, requiring no external components. [14].

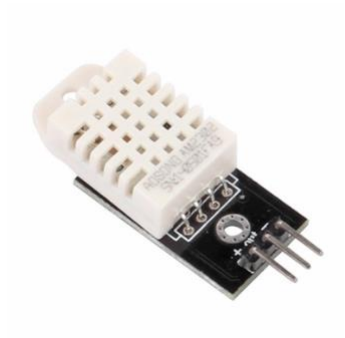
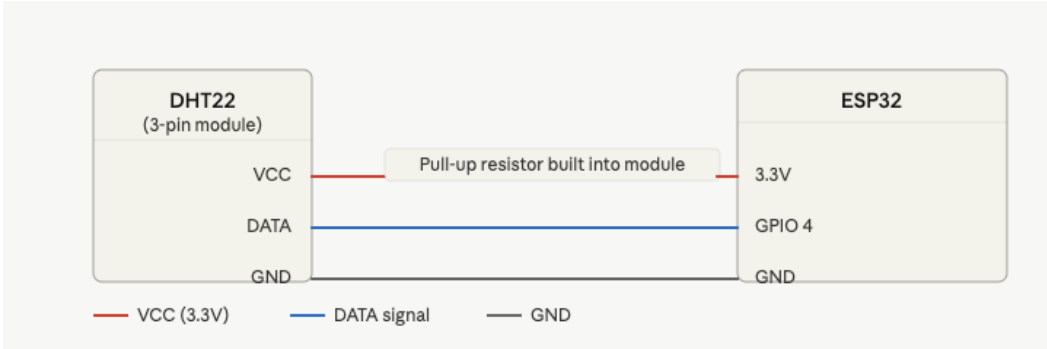


Figure 7.1 DHT22



Readings are captured using the DHTesp library, which provides a non-blocking interface for the one-wire protocol:

```
// Initialise DHT22
dht.setup(DHT_PIN, DHTesp::DHT22);
```

```
// DHT22 reading — only add temperature block if sensor returns valid data
// isnan() catches the NaN value DHTesp returns on a read failure.
TempAndHumidity data = dht.getTempAndHumidity();
if (!isnan(data.temperature) && !isnan(data.humidity)) {
  JsonObject temp    = doc.createNestedObject("temperature");
  temp["temperature_c"] = data.temperature;
  temp["humidity_pct"]  = data.humidity;
  Serial.printf("Temp: %.1f C  Humidity: %.1f%%\n", data.temperature, data.humidity);
} else {
  Serial.println("DHT22 read failed — sending WiFi only this cycle");
}
```

The getTempAndHumidity() call reads both values in a single operation. If the sensor is not yet warmed up or returns an invalid reading, the library returns NAN, and the firmware checks for this before including the values in the JSON payload. This ensures that no invalid readings are sent to the backend.

7.3 MQ-135 Air Quality Sensor

The MQ-135 is an electrochemical gas sensor capable of detecting a range of air pollutants including ammonia (NH_3), nitrogen oxides (NO_x), carbon dioxide (CO_2), benzene, alcohol, and smoke [15]. It operates by measuring the change in resistance of a tin dioxide (SnO_2) sensing element when exposed to target gases, converting this into an analog voltage output. Table 7.3.1 summarizes the key specifications.

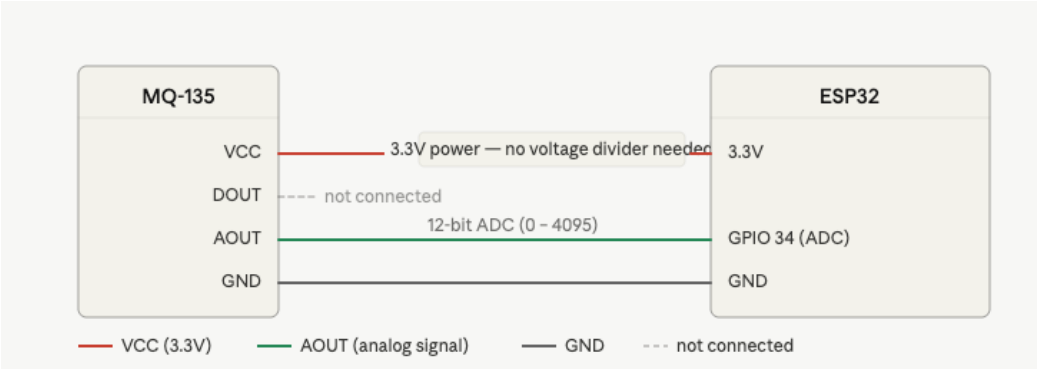
Table 7.2 MQ-135 Specifications

Parameter	Value
Target gases	NH_3, NO_x, CO_2, benzene, alcohol, smoke
Detection range	10–1000 ppm
Operating voltage	5V (used at 3.3V in this project)
Output	Analog voltage (0–VCC)
Warm-up time	Approximately 30 seconds
Output pin	AOUT → GPIO 34 (ESP32 ADC)

One design decision I made early in the project was not to implement the full MQ-135 ppm conversion formula, which involves calibrating the sensor's Rs/Ro resistance ratio in clean air and applying a logarithmic scaling curve. This calibration requires a controlled environment and a reference gas concentration, which was not practical within the scope of this project. Instead, the raw 12-bit ADC value from the ESP32 is used as a relative indicator of air quality. Because the heatmap colour thresholds are configurable, the same visual output can be achieved by mapping the raw ADC range to the Good, Moderate, and Poor bands, without needing an absolute ppm value. Both the raw value and a derived ppm estimate are stored in the database to allow proper calibration to be applied later without data loss.

The MQ-135 AOUT pin is connected directly to GPIO 34, which is one of the ESP32's dedicated input-only ADC pins. VCC is connected to the 3.3V rail rather than 5V, which slightly reduces the sensor's sensitivity but keeps the output voltage within the ESP32's 3.3V ADC input range, avoiding the need for a voltage divider [15].

Figure 7.2 MQ-135




```
int rawAir = analogRead(MQ135_PIN);
if (rawAir > 0) {
  JsonObject air   = doc.createNestedObject("air_quality");
  air["ppm"]       = rawAir;   // raw ADC used as relative PPM indicator
  air["raw_value"] = rawAir;
  Serial.printf("MQ-135 Raw ADC: %d\n", rawAir);
} else {
  Serial.println("MQ-135 read zero - omitting air_quality block this cycle");
}
```

The reading is only added to the payload when a positive value is returned, which guards against readings taken before the sensor has completed its warm-up period.

7.4 Wiring and Circuit Design

Table 7.3 summarizes the complete wiring connections between the sensors and the ESP32. All components share a common ground and are powered from the ESP32's 3.3V output pin.

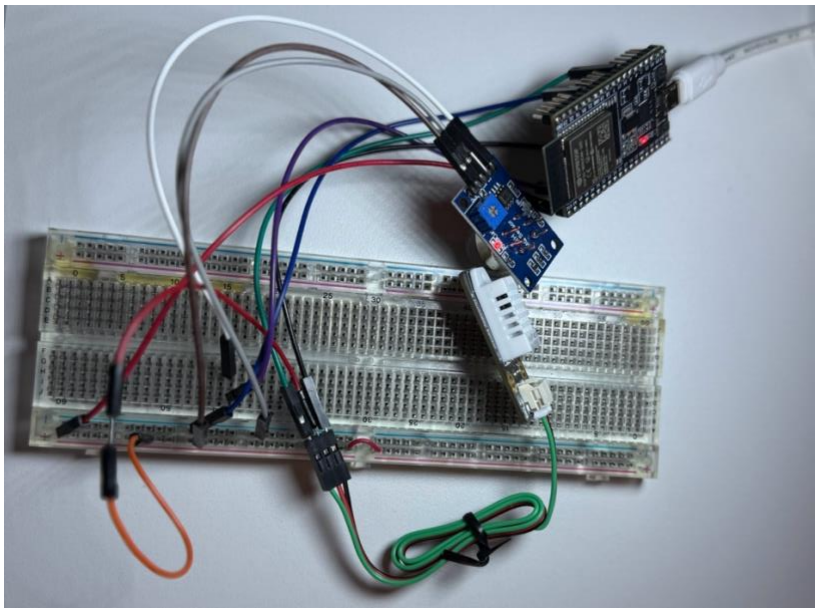


Figure 7.3 Full breadboard circuit

7.5 Power Supply

During development, the ESP32 is powered over USB from a laptop, which also provides the serial connection used for programming and debugging. For deployment in a building, the device is powered from a standard USB wall adapter providing 5V at a minimum of 500mA. The ESP32's onboard voltage regulator steps this down to 3.3V, which also powers both sensors. A USB power bank is a suitable alternative for locations that do not have a nearby wall socket, providing a fully wireless deployment with a runtime of several hours depending on the power bank capacity.

7.6 Problems Encountered

The most significant hardware problem encountered during the project was integrating the MQ-135 sensor alongside the DHT22 without one interfering with the other. Early in development, I found that calling the DHT22 read function and the analogRead for the MQ-135 in rapid succession caused occasional timing conflicts, particularly when the one-wire protocol

for the DHT22 required precise microsecond-level timing. The solution was to ensure the DHT22 read completed fully before triggering the ADC read, and to add a short delay between the two operations.

A second issue was that the MQ-135 readings were highly unstable in the first 30 seconds after power-on, returning values near zero or fluctuating erratically. This is consistent with the datasheet specification for a warm-up period. I addressed this by adding a startup delay and a guard condition that skips the air quality reading if the raw ADC value is below a minimum threshold, indicating the sensor has not yet stabilised.

8 Software : Backend

The backend is the central layer of the MSSIA architecture. It receives sensor data from the ESP32, validates it, persists it to the database, and exposes a REST API that the frontend consumes. It is written in Python using the FastAPI framework and runs as a permanent service on an AWS EC2 server.

8.1 FastAPI Framework

FastAPI is a modern Python web framework built on top of Starlette and Pydantic [16]. It was chosen for this project for three reasons. First, it generates OpenAPI documentation automatically from the route definitions, which made testing endpoints during development significantly faster. Second, it uses Python type hints for request and response validation, which catches malformed payloads before they reach the database. Third, it supports asynchronous request handling, which was important for the Server-Sent Events endpoint added later in the project.

The backend is structured as a single `main.py` file containing all route definitions, with models and schemas separated into `models.py` and `schemas.py`. SQLAlchemy is used as the Object-Relational Mapper (ORM), allowing database tables to be defined as Python classes rather than raw SQL, and queries to be written as Python method calls rather than SQL strings [17].

8.2 Database Design (PostgreSQL) and the SQLite to PostgreSQL Migration

The first version of the backend used SQLite as the database. SQLite is a file-based database that requires no server setup, making it a practical choice for early development when the priority was getting the data pipeline working rather than configuring infrastructure. During the initial sprint, the ESP32 was successfully sending JSON payloads to the FastAPI endpoint, and the data was being stored in a local SQLite file.

As the project grew, SQLite's limitations became apparent. It does not support concurrent writes well, which caused occasional errors when the ESP32 was sending data at the same time as the frontend was fetching it. It also does not enforce foreign key constraints by default, which meant referential integrity had to be managed manually in the application code rather than at the database level. For a system that was going to be deployed to the cloud and accessed by multiple users simultaneously, a proper relational database was necessary.

I migrated to PostgreSQL because it is a production-grade relational database that handles concurrent connections robustly, enforces foreign key constraints natively, and integrates cleanly with SQLAlchemy through the psycopg2 driver [18]. The migration was straightforward because SQLAlchemy abstracts the database engine, changing from SQLite to PostgreSQL required only updating the connection string in the configuration and installing the psycopg2 package. All table definitions and queries remained unchanged.

Figure 8.1 Database Schema open on the Server

```
(.venv) ubuntu@ip-172-31-43-221:~/Final-Year-Project/esp32-wifi-pipeline/FastAPI$ psql -U mssia_user -psql -U mssia_user -d mssia_db -h localhost -c "\dt"
Password for user mssia_user:
             List of relations
 Schema |      Name       | Type  |   Owner
--------+-----------------+-------+-----------
 public | building        | table | mssia_user
 public | dht22_reading   | table | mssia_user
 public | floor_plan      | table | mssia_user
 public | mq135_reading   | table | mssia_user
 public | room            | table | mssia_user
 public | scan_point      | table | mssia_user
 public | wifi_scan       | table | mssia_user
(7 rows)
```

Table 8.1 Database Schema

Table	Purpose	Key Columns
building	Represents a physical building	id, name, description
floor_plan	A floor plan image within a building	id, building_id, floor_name, image_url
scan_point	A named location on a floor plan	id, floorplan_id, x, y (0.0–1.0), label, assigned_node
wifi_scan	Wi-Fi RSSI readings per scan cycle	id, scan_point_id, ssid, bssid, rssi, channel, enc, received_at
dht22_reading	Temperature and humidity readings	id, scan_point_id, temperature_c, humidity_pct, received_at
mq135_reading	Air quality readings	id, scan_point_id, ppm, raw_value, received_at

Scan point coordinates are stored as normalized floating-point values between 0.0 and 1.0, representing proportional positions on the floor plan image. This ensures that the coordinates remain valid regardless of how the floor plan image is displayed, whether on a large desktop monitor or a small mobile screen because the frontend converts them to pixel positions by multiplying by the actual rendered image dimensions at display time.

8.3 API Endpoints

The backend exposes a set of REST endpoints that the frontend consumes for both the admin studio and the public user viewer. Table 8.2 summarizes the main endpoints.

Table 8.2 Endpoint Summary

Method	**Endpoint**	Purpose
POST	/ingest	Receive sensor data from ESP32
POST	/auth/login	Admin login, returns JWT
GET	/auth/verify	Validate a JWT token
GET	/buildings	List all buildings
POST	/buildings	Create a building
GET	/floorplans/{id}	Get a floor plan
PATCH	/floorplans/{id}	Rename a floor plan
GET	/heatmap/floorplan/{id}	Latest Wi-Fi readings per scan point
GET	/heatmap/floorplan/{id}/dht22	Latest DHT22 readings per scan point
GET	/heatmap/floorplan/{id}/mq135	Latest MQ-135 readings per scan point
GET	/scan-points/{id}/wifi-history	Historical Wi-Fi readings for a point
GET	/devices	List all scan points
GET	/devices/known	List assigned nodes
GET	/alerts	Floor-level environmental alerts
GET	/events/floorplan/{id}/heatmap	Server-Sent Events stream for live updates

8.4 JWT Authentication

The admin interface requires authentication to prevent unauthorised users from modifying floor plans or device assignments. Authentication is implemented using JSON Web Tokens

(JWT), a widely used open standard for securely transmitting claims between parties as a signed JSON object [19].

When an admin submits their credentials to POST /auth/login, the backend verifies the username and password against environment variables stored in a .env file on the server. If the credentials are correct, the backend generates a signed JWT using the HS256 algorithm with a secret key, sets an expiry of eight hours, and returns the token to the frontend. The frontend stores this token in localStorage and includes it in the Authorization: Bearer header on every subsequent admin request. The GET /auth/verify endpoint allows the frontend to check whether a stored token is still valid on page load, redirecting to the login page if it has expired.

Credentials are stored in a .env file on the server and loaded using python-dotenv [20]. The .env file is excluded from the GitHub repository via .gitignore, ensuring that no sensitive credentials are ever committed to version control.

8.5 Data Ingestion and Validation

The primary entry point for sensor data is the `POST /ingest` endpoint. When a POST request arrives, the endpoint first checks whether the `node` field in the payload matches a registered, assigned device in the `scan_point` table. If the node name is not found or has not been assigned to a scan point, the request is rejected with an HTTP 403 Forbidden response. This design ensures that only authorised, physically placed devices can write to the database, preventing accidental or malicious data from entering the system.

```python
@app.post("/ingest")
def ingest(
    payload: Dict[str, Any] = Body(...),
    db: Session = Depends(get_db),
):
    node        = payload.get("node")
    scans       = payload.get("scans", [])
    temperature = payload.get("temperature")   # optional DHT22 block

    # — Validation ————————————————————————————————————————
    if not node:
        raise HTTPException(status_code=400, detail="Missing 'node' field in payload")

    if not isinstance(scans, list) or not scans:
        raise HTTPException(status_code=400, detail="Missing or empty 'scans' array in payload")

    # — Node validation (security) ————————————————————————
    known = db.query(ScanPointDB).filter(ScanPointDB.assigned_node == node).first()
    if not known:
        raise HTTPException(status_code=403, detail="Unassigned")

    # — Stamp server-side timestamp once for this batch ————————————
    server_now    = datetime.now(timezone.utc)
    scan_point_id = known.id
```

If the node is valid, the endpoint iterates over the Wi-Fi scan array in the payload and inserts a `wifi_scan` row for each access point found. If the payload also contains a `temperature` block, a `dht22_reading` row is inserted. If it contains an `air_quality` block, an `mq135_reading` row is inserted. All timestamps are applied server-side using UTC, rather than relying on a timestamp from the ESP32. This is because the ESP32 has no real-time clock and `millis()` resets on every reboot, making device-side timestamps unreliable.

8.6 Real-Time Updates (Server-Sent Events)

The heatmap dashboard updates in real time when new sensor data arrives from the ESP32 without requiring the user to refresh the page. This is achieved using Server-Sent Events (SSE), a standard web technology that allows a server to push data to a browser over a persistent HTTP connection [21].

The `GET /events/floorplan/{id}/heatmap` endpoint is an asynchronous generator that polls the database every three seconds for the most recent ingest timestamp on a given floor plan. When the timestamp changes, indicating that new data has arrived, it emits a message event containing the new timestamp. The frontend's `EventSource` object receives this event and re-fetches the full heatmap data.

SSE was chosen over WebSockets because it requires only a one-way data stream from server to client. WebSockets are bidirectional and introduce additional complexity in connection

management that was unnecessary for this use case. SSE integrates naturally with HTTP and works through standard reverse proxies, making it simpler to implement and maintain [22].

One important configuration detail is the `X-Accel-Buffering: no` response header, which instructs Nginx to disable response buffering for this endpoint. Without this header, Nginx holds SSE frames in its buffer until a threshold is reached, causing events to arrive in delayed batches rather than in real time.

8.7 Environmental Alerts

The `GET /alerts` endpoint computes floor-level environmental alerts by aggregating the most recent sensor readings across all scan points on each floor plan. Rather than alerting on individual device connectivity, the endpoint answers a more operationally useful question: is the environment on this floor within acceptable thresholds?

Four alert types are computed: poor air quality (triggered when more than 50% of scan points report a poor ADC reading, becoming critical at 75%), poor Wi-Fi signal (same threshold logic), high humidity (floor average above 70%), and low humidity (floor average below 30%). The percentage-based thresholds for air quality and signal strength mean that a single malfunctioning sensor does not trigger a critical alert, reducing false positives.

8.8 Problems Encountered

The most significant problem encountered during backend development was the original database design for linking Wi-Fi scan readings to rooms. The initial architecture required three tables to record a single scan: `wifi_scan` for the raw ESP32 readings, `scan_session` to track a manually started and stopped scanning session, and `room_scan` as a junction table linking each `wifi_scan` row to a `room_id` and `session_id`. To associate a scan with a room, the workflow required calling a `/start-scan` endpoint before the ESP32 began sending data, waiting for the scan to complete, and then calling `/stop-scan` to close the session. The heatmap query then had to perform a three-table JOIN just to answer the question "what is the average signal strength in Room 3?"

The problem became clearer on closer inspection: the `wifi_scan` table already had a `room_id` column on it. It was simply always NULL because the room association was being stored in `room_scan` rather than directly on the scan row. The three-table design added significant complexity without providing any benefit over simply stamping the `room_id` directly onto each scan at ingest time.

The solution was to remove both the `scan_session` and `room_scan` tables entirely and replace the session lifecycle with a simpler in-memory active room mechanism. A `POST /rooms/{id}/set-active` endpoint sets an `active_room_id` variable on the FastAPI application instance. At the `/ingest` endpoint, each incoming scan is then stamped with the current `active_room_id` directly on the `wifi_scan` row. When scanning is finished, `POST /rooms/clear-active` resets the variable. The heatmap query reduced from a three-table JOIN down to a direct two-table JOIN, and the `session_id` parameter was removed from the heatmap endpoint entirely. The files changed were `models.py`, `main.py`, `schemas.py`, and the Next.js API type definitions and fetch helpers.

This was the most challenging design problem in the project because it required stepping back from the existing implementation and recognising that the complexity was not serving the system's actual needs. The fix was ultimately straightforward, but identifying that the original design was the problem rather than the query logic or the frontend took considerable time and was resolved with the help of an AI assistant.

A second problem arose during the migration from SQLite to PostgreSQL. Several SQLAlchemy relationship definitions that had worked correctly under SQLite began raising integrity errors under PostgreSQL because PostgreSQL enforces foreign key constraints immediately rather than lazily. Each cascade delete behaviour had to be explicitly defined across the `scan_point`, `wifi_scan`, `dht22_reading`, and `mq135_reading` tables to ensure that deleting a scan point correctly removed all associated readings without leaving orphaned rows.

9 Software: Frontend

The frontend is a Next.js 16 application written in TypeScript, deployed on Vercel and accessible at final-year-project-alpha-azure.vercel.app. It is the layer of the system that both administrators and general users interact with directly. The application is divided into two distinct areas: an authenticated Admin Studio for managing the system configuration, and a public User Viewer for browsing buildings and viewing live heatmaps.

9.1 Next.js Architecture

Next.js is a React framework built on top of Node.js that provides server-side rendering, file-based routing, and an integrated build system [23]. Version 16 was used in this project, with Turbopack as the development bundler, which provided significantly faster hot-reload times during development compared to the default Webpack bundler. The application is written entirely in TypeScript, which provided compile-time type checking across all API response shapes, component props, and state variables.

The project follows Next.js's App Router convention, where each folder inside the app/ directory corresponds to a URL route. The two main routes are app/admin/ for the Admin Studio and app/user/ for the public viewer. Shared components are stored in components/, API functions in lib/api/, and custom React hooks in lib/. This separation of concerns kept the codebase organised as features were added incrementally across multiple sprints.

All API calls to the FastAPI backend are centralised in the lib/api/ directory, with separate files for each resource type: buildings.ts, floorplans.ts, heatmap.ts, dht22.ts, mq135.ts, and alerts.ts. These are re-exported through a single lib/api/index.ts file so that components import from one location regardless of which resource they need. This pattern meant that when an endpoint URL or response shape changed on the backend, only one file in the frontend needed to be updated.

9.2 Admin Studio

The Admin Studio is the configuration interface for the system. It is protected by a JWT authentication guard that runs on component mount: if no valid token is found in `localStorage`, the user is redirected to the login page before any admin data is loaded. The

login page calls `POST /auth/login` on the FastAPI backend and stores the returned token in `localStorage` for subsequent requests.

Once authenticated, the Admin Studio provides four main functions. First, the admin can create buildings and upload floor plan images, which are stored on the server and referenced by URL in the database. Second, scan points can be placed by clicking directly on the floor plan image and each click records the position as a normalized coordinate pair (x, y) between 0.0 and 1.0, which is stored in the database and remains valid regardless of how the image is later displayed. Third, registered ESP32 devices can be assigned to scan points, linking the physical hardware location to a named position on the map. Fourth, buildings and floor plan names can be renamed directly from the breadcrumb navigation using a `PATCH /floorplans/{id}` endpoint that was added specifically for this purpose, since the existing `PUT` endpoint required the full floor plan object including the image URL.

9.3 User Viewer

The User Viewer is the public-facing interface and requires no login. Users can browse available buildings, select a floor plan, and view a live heatmap overlaid on the floor plan image. A mode selector at the top of the viewer allows switching between the four heatmap types: Signal, Temperature, Humidity, and Air Quality. Each mode fetches data from a different backend endpoint and renders the scan point positions as colour-coded blobs. Clicking on any blob opens a slide-in detail panel on the right side of the screen showing the current readings and a time-range chart of historical data.

The User Viewer was built as a separate component from the Admin Studio's view mode rather than reusing the same component, because the two have different interaction models. The admin view uses a popup-style panel, which was found during testing to be too small for displaying charts clearly. The user view uses a wider slide-in panel that gives more space to the historical charts and overview data, and this component was later reused inside the admin view mode as well, replacing the original popup entirely.

9.4 Heatmap Rendering

The heatmap is rendered by overlaying absolutely positioned circular blobs on top of the floor plan image. Each blob's position is calculated by converting the stored normalized coordinates to pixel values using the actual rendered dimensions of the image on screen. The colour of each blob is determined by mapping the sensor reading to one of four threshold bands, which are defined differently for each heatmap mode. Table 9.1 summarizes the colour thresholds for each mode.

Table 9.1 Heatmap Colour Thresholds

Mode	Green	Amber	Orange	Red
Signal (RSSI dBm)	≥ −50	−50 to −60	−60 to −70	< −70
Temperature (°C)	16–21	< 16	—	> 21
Humidity (% RH)	30–60	< 30	—	> 60
Air Quality (ADC)	< 2000	2000–2800	—	> 2800

No third-party charting library is used. All charts including the signal history line chart, the temperature history chart, the humidity chart, and the air quality chart are rendered using custom SVG elements generated directly in React. This approach was taken because the available charting libraries either introduced significant bundle size or required complex configuration to match the visual style of the dashboard. Drawing SVG directly gave full control over appearance, interactivity, and tooltip formatting.

9.5 Floor Plan Interaction (Zoom, Pan, Pin Placement)

Three custom React hooks manage the interactive behaviour of the floor plan components. The `useImageBounds` hook computes the actual rendered position and dimensions of the floor plan image after it loads, accounting for the `objectFit: contain` CSS property which scales the image to fit its container while preserving the aspect ratio. This creates letterbox bars on the sides or top of the container when the container's proportions do not match the image's proportions. Without this hook, scan point positions calculated as percentages of the container would be offset by the amount of the letterbox, causing pins and blobs to appear in the wrong

locations. The hook attaches a `ResizeObserver` to the container so that positions are recomputed automatically whenever the window is resized or the panel layout changes. The `useZoomPan` hook manages scale and pan state for the floor plan canvas, allowing users to zoom in with the mouse wheel and pan by dragging. The `useAlerts` hook polls the `/alerts` endpoint every thirty seconds and exposes the current alert state to the Admin Studio top bar.

9.6 Data Export (CSV and PowerPoint)

The Admin Studio includes an export feature that allows the admin to download a summary of the current floor plan's sensor data. Two export formats are supported. The CSV export fetches the latest readings for all scan points across all four sensor modes, joins them into a single table, and triggers a browser download using a dynamically created anchor element. The PowerPoint export generates a two-slide deck using the `pptxgenjs` library [24]: the first slide is a dark title slide with summary statistics (total scan points, assigned, unassigned), and the second slide is a colour-coded data table using the same signal and air quality thresholds as the heatmap.

9.7 Problems Encountered

The most significant frontend problem was the scan point pin misalignment bug. Pins placed by the admin in Edit mode and heatmap blobs shown in View mode were appearing at visually incorrect positions, with the offset changing unpredictably when the window was resized or when switching between modes. The root cause was that the floor plan image was using `objectFit: contain`, which does not fill the full container . It adds letterbox padding, but the pin positions were being calculated as percentages of the full container width rather than the rendered image width. The fix required the `useImageBounds` hook, which uses the image's natural dimensions and the container's current size to compute the exact rendered dimensions and offset using the same algorithm the browser uses for `objectFit: contain`. The hook also attaches a `ResizeObserver` so that these values are recomputed on every layout change, keeping pins correctly positioned at all times.

A second problem involved the zoom and pan feature. After implementing the `useZoomPan` hook, dragging a pin to a new position while the canvas was zoomed caused the pin to snap to

the wrong location. The cause was a stale closure: the drag event handler was capturing the scale value at the time the drag started, not the current scale. Because React state does not update closed-over variables, the handler was always using an outdated scale. The solution was to store the current scale and pan values in a `useRef` object that is updated on every render and have the drag handler read from the ref rather than from closed-over state.

A third problem arose when trying to implement the PowerPoint export using `pptxgenjs`. Importing the library at the top of the file caused the Next.js build to fail with a `window is not defined` error, because Next.js performs server-side rendering during the build process and `pptxgenjs` accesses browser-specific APIs at module load time. The fix was to use a dynamic `import()` statement inside the async export function, deferring the import until the function is called in the browser at runtime.

A fourth problem was encountered when uploading floor plan images in .jpeg format. Files with the .jpeg extension were not appearing in the macOS file picker despite image/jpeg being listed in the accept attribute. On macOS, the operating system's file picker filters files by extension rather than MIME type, so .jpeg files were hidden because the extension was not explicitly listed. The fix was to update the accept attribute to include both MIME types and explicit extensions: accept="image/png,image/jpeg,.png,.jpg,.jpeg". A related issue in the same component was that upload errors were caught in a catch block that only called console.error(), meaning any backend rejection was swallowed silently with no message shown to the user. An uploadFloorError state variable was added and the catch block was updated to set it, displaying the error message inside the upload modal so the user could see exactly what went wrong.

10 Cloud Deployment

Before this sprint, MSSIA could only run locally on a development laptop. The objective of the cloud deployment was to make the full system permanently accessible over the internet so that the ESP32 hardware could send data to a stable URL, and any user could open the dashboard from any browser without the development machine being present. This section describes each layer of the deployed infrastructure, the decisions made at each stage, and the problems encountered along the way.

10.1 Infrastructure Overview

The deployed system consists of two hosting environments. The FastAPI backend and PostgreSQL database are hosted together on a single AWS EC2 virtual machine. The Next.js frontend is hosted on Vercel. These two environments are connected through a public HTTPS API served from the EC2 server via a DuckDNS domain. Figure 10.1 illustrates the full deployment architecture including the DNS chain, the Nginx reverse proxy layer, and the request flows from both the ESP32 hardware and the browser.

Figure 10.1 Deployment Architecture Diagram

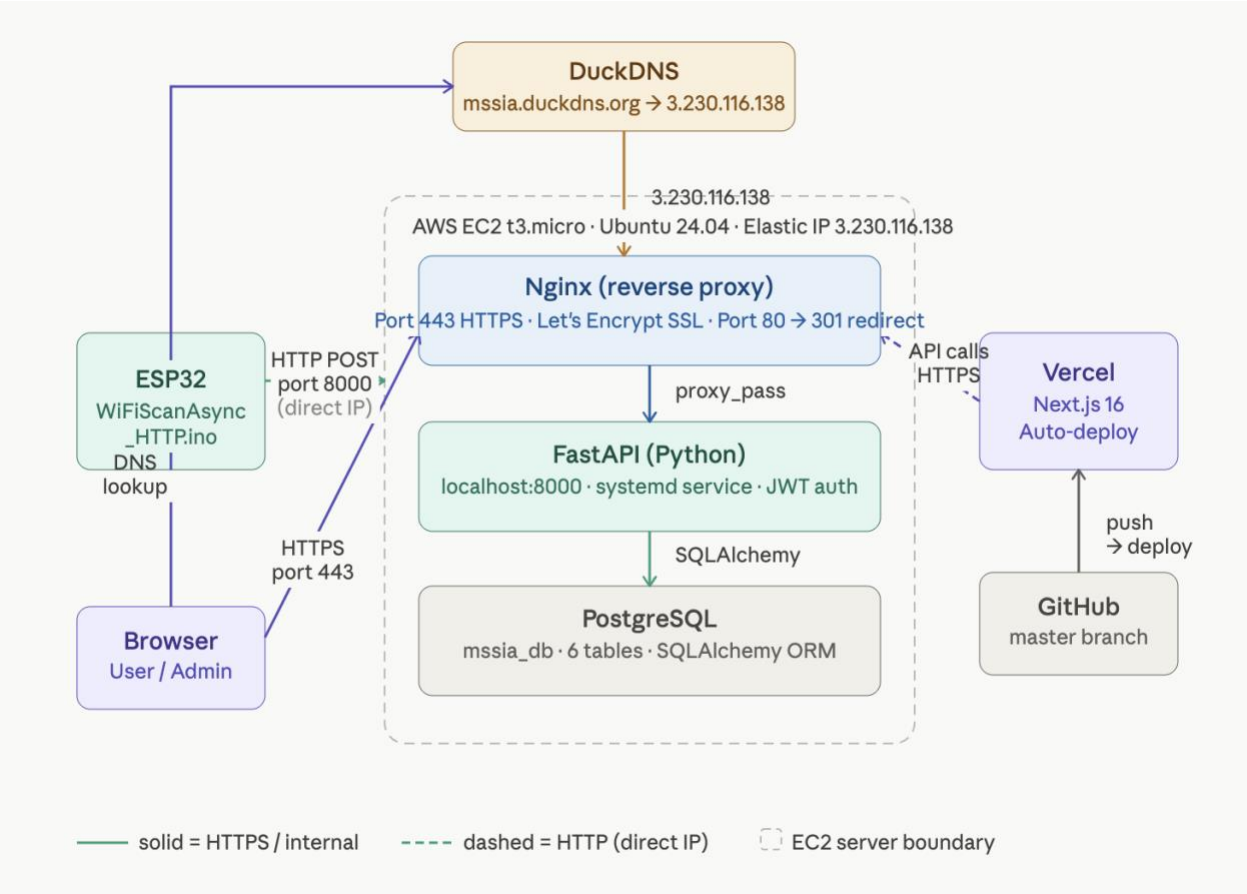


Figure 10.1 Cloud deployment architecture showing the DNS chain from DuckDNS to the EC2 Elastic IP, the Nginx HTTPS reverse proxy, the FastAPI service, and the Vercel frontend.

10.2 AWS EC2 Setup

Amazon Web Services Elastic Compute Cloud (EC2) provides resizable virtual machines in the cloud [25]. A t3.micro instance running Ubuntu 24.04 LTS was selected for this project. The t3.micro tier is eligible for the AWS Academy free tier allocation, making it a cost-effective choice for my project. Ubuntu 24.04 was chosen as the operating system because it is the current long-term support release and has first-class support for the Python, Nginx, and PostgreSQL packages required by the project.

After launching the instance, an Elastic IP address was assigned to it. An Elastic IP is a static public IP address that remains associated with the EC2 instance regardless of whether the instance is stopped and restarted [26]. Without an Elastic IP, the instance would receive a new random public IP address every time it restarted, which would break the DuckDNS DNS record and the ESP32's hardcoded server URL.

The following packages were installed on the server after the initial connection was established:

```bash
sudo apt update
sudo apt install -y python3 python3-pip python3-venv git nginx
sudo apt install -y postgresql postgresql-contrib
```

The PostgreSQL service was enabled and started with systemd, and the `mssia_db` database and `mssia_user` account were created with the same credentials as the local development environment. SQLAlchemy creates all required tables automatically when FastAPI starts, so no manual SQL migration scripts were required.

10.3 FastAPI as a system Service

Running FastAPI manually in a terminal is not suitable for a production deployment because the process stops as soon as the terminal session is closed. To make FastAPI run permanently and restart automatically on crash or reboot, a systemd service unit file was created at `/etc/systemd/system/mssia.service`. Systemd is the standard Linux service manager and is responsible for starting, stopping, and monitoring background processes on Ubuntu [27].

```
(.venv) ubuntu@ip-172-31-43-221:~/Final-Year-Project/esp32-wifi-pipeline/FastAPI$ cat /etc/systemd/system/mssia.service
[Unit]
Description=MSSIA FastAPI
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/Final-Year-Project/esp32-wifi-pipeline/FastAPI
Environment="PATH=/home/ubuntu/Final-Year-Project/esp32-wifi-pipeline/FastAPI/.venv/bin"
EnvironmentFile=/home/ubuntu/Final-Year-Project/esp32-wifi-pipeline/FastAPI/.env
ExecStart=/home/ubuntu/Final-Year-Project/esp32-wifi-pipeline/FastAPI/.venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

The `Restart=always` directive ensures that if the FastAPI process crashes for any reason, systemd automatically restarts it within seconds. The `After=network.target` directive ensures the service does not start until the network interfaces are ready, preventing startup failures on reboot.

10.4 Nginx Reverse Proxy and HTTPS

Nginx is a high-performance web server and reverse proxy [28]. In this deployment, it serves two purposes: terminating HTTPS connections using a Let's Encrypt SSL certificate, and forwarding decrypted requests to FastAPI running on `localhost:8000`. This means that external traffic always uses HTTPS on port 443, while FastAPI only ever sees plain HTTP traffic on the local loopback interface.

The Nginx server block configuration routes all incoming requests for `mssia.duckdns.org` to the FastAPI backend:

```
(.venv) ubuntu@ip-172-31-43-221:~/Final-Year-Project/esp32-wifi-pipeline/FastAPI$ cat /etc/nginx/sitescat /etc/nginx/sites-available/mssia
server {
    server_name mssia.duckdns.org;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/mssia.duckdns.org/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/mssia.duckdns.org/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot

}
server {
    if ($host = mssia.duckdns.org) {
        return 301 https://$host$request_uri;
    } # managed by Certbot


    listen 80;
    server_name mssia.duckdns.org;
    return 404; # managed by Certbot
```

The second server block redirects all HTTP traffic on port 80 to HTTPS with a 301 permanent redirect, ensuring that all communication is encrypted. The SSL certificate was issued by Let's Encrypt using Certbot, a free and automated certificate authority [29]. Certbot configures automatic certificate renewal, so the certificate never expires without intervention.

HTTPS was a requirement rather than an optional enhancement. The Vercel frontend is served over HTTPS by default, and modern browsers enforce a Mixed Content policy that blocks HTTP requests made from HTTPS pages [30]. Without HTTPS on the backend, the admin login and all API calls from the frontend would be silently blocked.

10.5 DuckDNS Domain

DuckDNS is a free dynamic DNS service that allows a domain name to be mapped to an IP address [31]. The subdomain `mssia.duckdns.org` was registered and pointed at the EC2 Elastic IP address `3.230.116.138`. This provides a stable, human-readable URL for the API rather than requiring users and the ESP32 to use a raw IP address. The DuckDNS record is a standard DNS A record that resolves the domain name to the Elastic IP.

10.6 Vercel Frontend Hosting

Vercel is a cloud platform designed specifically for hosting Next.js applications [32]. I chose it over AWS Amplify because AWS Academy accounts do not have permission to create the IAM roles that Amplify requires during its setup process. Vercel requires no IAM configuration and connects directly to the GitHub repository.

The project repository is a monorepo containing both the FastAPI backend and the Next.js webapp. Vercel was configured to use `esp32-wifi-pipeline/webapp` as the root directory so that only the frontend is built and deployed. The environment variable `NEXT_PUBLIC_API_URL=https://mssia.duckdns.org` was set in the Vercel project settings rather than in a `.env` file, which ensures the variable is available during the build process and is never committed to version control.

Vercel automatically triggers a new deployment whenever a commit is pushed to the master branch on GitHub. This means that any frontend change deployed to GitHub is live within minutes without any manual deployment steps on the server.

10.7 DNS Chain and Request Flow

The full DNS chain and request flow for a typical frontend API call is as follows. When a user opens the dashboard in a browser, the Next.js JavaScript running in their browser makes an HTTP GET request to `https://mssia.duckdns.org/heatmap/floorplan/1`. The browser performs a DNS lookup, asking the DuckDNS nameservers to resolve `mssia.duckdns.org`. DuckDNS returns the Elastic IP address `3.230.116.138`. The browser opens a TLS connection to that IP on port 443. Nginx on the EC2 server receives the connection, verifies the Let's Encrypt certificate, decrypts the request, and forwards it as plain HTTP to FastAPI on `localhost:8000`. FastAPI queries PostgreSQL, assembles the response, and returns it through Nginx to the browser.

The ESP32 uses a different path. Rather than sending data to `mssia.duckdns.org`, it sends directly to `http://3.230.116.138:8000/ingest` using the raw Elastic IP and port. This is necessary because the ESP32's `HTTPClient` library does not follow HTTP 301 redirects. If the ESP32 were to use the DuckDNS domain, Nginx would respond with a 301 redirect from HTTP to HTTPS, and the ESP32 would interpret this as an error rather than following the redirect to the correct URL.

10.8 Problems Encountered

The first problem encountered was an SSH connection timeout immediately after the EC2 instance was launched. The instance showed as running in the AWS Management Console, but all SSH connection attempts timed out. The cause was that the EC2 security group had the inbound SSH rule set to a specific IP address that no longer matched the current network. Home broadband connections use dynamic IP addresses that change periodically, so the rule had become outdated. The fix was to update the SSH inbound rule source to `0.0.0.0/0`, allowing connections from any IP address. Authentication security is maintained by the `.pem` private key requirement regardless of the source IP.

The second problem was the initial attempt to deploy the frontend using AWS Amplify. The Amplify setup process requires creating an IAM logging role automatically during configuration. AWS Academy accounts have restricted IAM permissions and cannot create roles, so the process failed immediately with an authorization error. I switched to Vercel, which has no IAM requirements and is the standard deployment platform for Next.js projects.

The third problem occurred after switching to Vercel. The first build failed because the `next.config.js` rewrite rule used `process.env.NEXT_PUBLIC_API_URL` as the destination URL. During the Vercel build process, environment variables are not yet available, so the destination resolved to an empty string, which is invalid. The fix was to add a fallback value, so the rewrite always has a valid destination even when the environment variable is absent at build time.

A fourth problem was encountered after HTTPS was set up and the frontend was deployed. The admin login was still failing silently. Inspecting the browser console revealed a CORS error with the origin listed as a Vercel preview URL that was not in the FastAPI `allow_origins` list. Vercel generates a unique preview URL for every deployment, meaning the origin changes with each push to GitHub. Listing individual preview URLs would require updating the server after every deployment. The solution was to replace the list of fixed origins in the FastAPI CORS middleware with a regex pattern that matches any Vercel preview URL for the project.

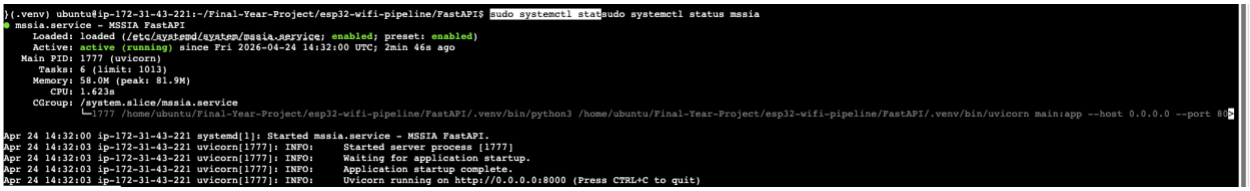
```
allow_origin_regex=r"https://final-year-project-.*\.vercel\.app"
```

A fifth problem was encountered when uploading floor plan images from a phone camera. Nginx was rejecting the upload with a 413 Request Entity Too Large error before the request ever reached FastAPI. Nginx's default client_max_body_size is 1 MB, but real-world floor plan photos are typically between 3 and 15 MB. The initial fix placed the client_max_body_size 20M directive in the server { } block in /etc/nginx/sites-available/default, which only covers HTTP traffic on port 80. The application is served over HTTPS on port 443, which is a separate server block, so the directive had no effect on live traffic. Moving the directive to the top-level http { } block in /etc/nginx/nginx.conf applied it globally to all server blocks regardless of port or

protocol, resolving the error. The configuration was validated with sudo nginx -t before reloading to prevent a misconfigured directive from taking the server offline.

11 Testing and Evaluation

Testing was carried out across four distinct areas: backend unit tests using pytest, frontend unit tests using Jest, end-to-end integration tests covering the full pipeline from ESP32 to database to browser, and manual hardware tests for each physical sensor. This section describes the approach taken in each area, the specific tests written, and the results obtained.



11.1 Backend Unit Tests (pytest)

The backend test suite is located in `esp32-wifi-pipeline/FastAPI/test_api.py` and is run using pytest. The tests are integration tests rather than unit tests they make real HTTP requests to a running FastAPI server and assert on the HTTP status codes and response body shapes. This approach was chosen because the most valuable thing to verify in this project is that the live endpoints behave correctly end to end, not that individual functions return expected values in isolation.

The test suite is organised into eight test classes, each covering a group of related endpoints. Table 11.1 summarizes the coverage.

Table 11.1 pytest Test Suite Coverage

Test Class	Endpoints Tested	Tests
TestHealth	GET /health	**2**
TestAuth	POST /auth/login, GET /auth/verify	**6**
TestIngest	POST /ingest	**4**
TestBuildings	GET /buildings, POST /buildings, GET /buildings/{id}	**4**
TestHeatmap	GET /heatmap/floorplan/{id}	**2**

TestWifiHistory	GET /scan-points/{id}/wifi-history	**3**
TestDevices	GET /devices, GET /devices/known	**3**
TestRooms	GET /rooms, DELETE /rooms/{id}	**2**
Total		**26**

To prevent confusing test output when the server is not running, a `conftest.py` file was created with a `pytest_collection_modifyitems` hook. This hook performs a pre-flight `GET /health` request before any tests are collected. If the server is unreachable, every test is automatically marked as skipped with a human-readable message rather than producing 26 connection error tracebacks. This is a standard pattern for integration test suites that depend on an external service being available.

The tests can be run against both the local development server and the live cloud deployment:

```bash
# Local
pytest test_api.py -v

# Against live cloud server
BASE_URL=https://mssia.duckdns.org pytest test_api.py -v
```

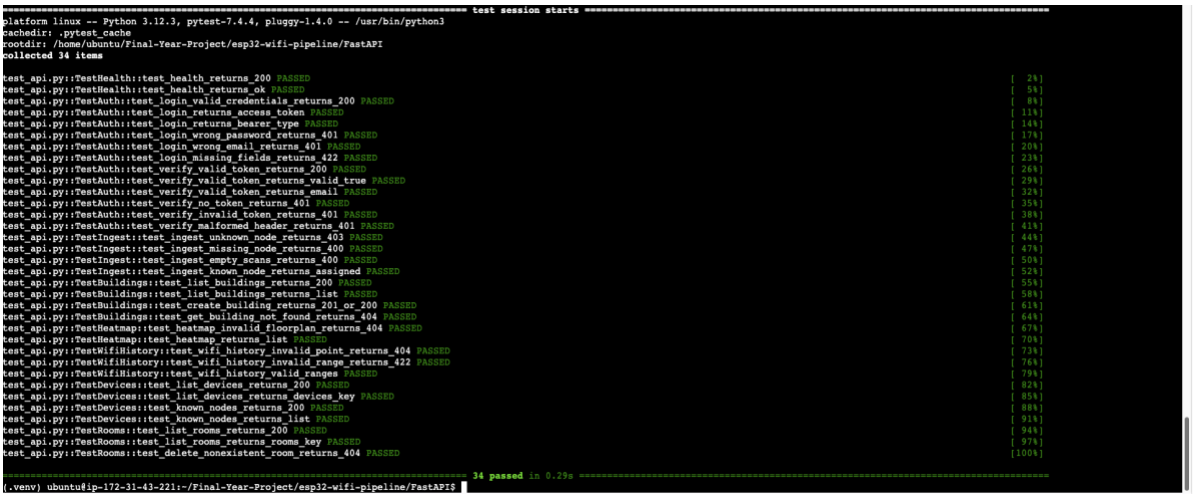


Figure 11.1 pytest test results screenshot

11.2 Frontend Unit Tests (Jest)

The frontend test suite is located in `esp32-wifi-pipeline/webapp/__tests__/api.test.ts` and is run using Jest with the ts-jest TypeScript preprocessor. Like the backend tests, these are integration tests that make real HTTP requests from the browser's fetch API and assert on response shapes.

The frontend suite covers nine test groups and includes 28 tests in total. It provides a separate confidence layer from the pytest suite: if the pytest tests pass but the Jest tests fail, the most likely cause is a CORS misconfiguration, a missing environment variable on the Vercel build, or a TypeScript type mismatch between what the backend sends and what the frontend expects.

```bash
# Local
npx jest

# Against live cloud server
NEXT_PUBLIC_API_URL=https://mssia.duckdns.org npx jest
```

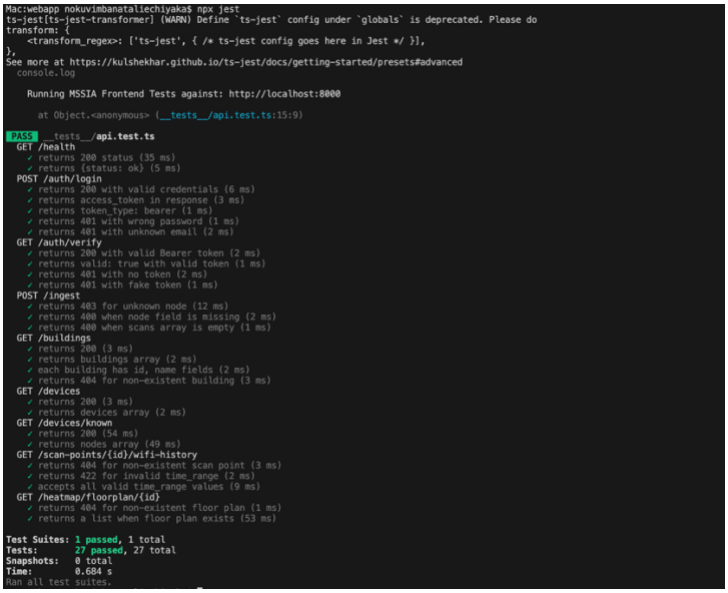


Figure 11.2 Jest test results screenshot

Figure 11.2 Jest test output showing all 28 tests passing against the live cloud server.

11.2.1 Problems Encountered

Setting up the Jest environment required resolving two configuration errors that are documented in Section 11.2. The first was that the `jest.config.ts` file required `ts-node` to parse, which was resolved by replacing it with a plain `jest.config.js` CommonJS file. The second was a `SecurityError` caused by Next.js 16 instrumentation triggering Jest 29's `localStorage` getter during environment construction, which was resolved by creating a custom Jest environment class that overrides `localStorage` in the constructor before the getter can fire.

11.3 End-to-End Testing — Playwright

End-to-end testing was carried out using Playwright v1.59, an open-source browser automation framework that controls a real Chromium browser programmatically and makes assertions against what appears on screen [33]. Unlike the pytest and Jest suites, which test the API layer directly, Playwright tests the complete system from the user's perspective — navigating pages, clicking buttons, filling forms, and asserting that the correct content is visible in the browser. This provides a confidence layer that neither the backend nor frontend tests alone can provide, because it verifies that the frontend, backend, authentication, and database are all working correctly as a complete pipeline.

Playwright was installed in the `webapp` directory and configured via `playwright.config.ts` to run all tests against the live Vercel deployment at `final-year-project-alpha-azure.vercel.app` using a headless Chromium browser with a 30-second per-test timeout. A `.env.test` file stores the admin credentials and base URLs, which are loaded into the config at runtime. The `.env.test` file is excluded from version control via the existing `.gitignore` rule.

Fifteen tests were written across three groups. The User Viewer group contains eight tests that verify the public-facing interface without requiring any login. The Admin group contains six tests that verify the full authentication flow from redirect through login to logout. The API Health group contains one test that navigates directly to the FastAPI health endpoint and asserts the JSON response.

Table 11.3 Playwright E2E Test Suite

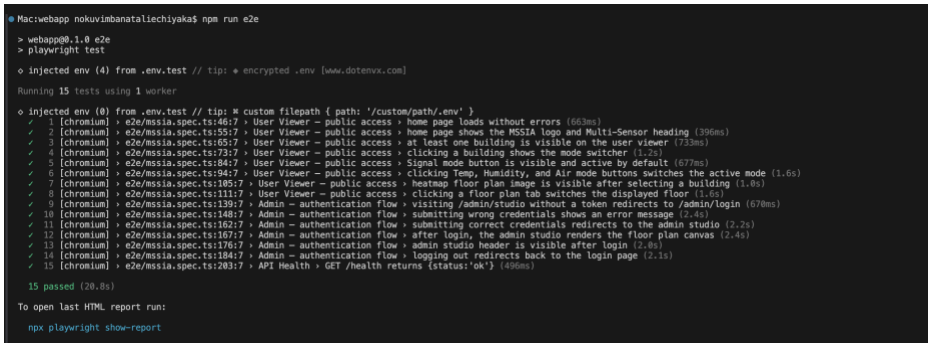
#	Test Name	Group	Expected Result
1	Home page loads without errors	User Viewer	Navigates to `/`, confirms MSSIA logo visible
2	Home page shows MSSIA logo and Multi-Sensor heading	User Viewer	"Multi-Sensor" and "Spatial Intelligence" headings visible
3	At least one building is visible on the user viewer	User Viewer	Navigates to `/user`, building button visible after API load
4	Clicking a building shows the mode switcher	User Viewer	Signal / Temp / Humidity / Air mode bar appears
5	Signal mode button is visible and active by default	User Viewer	All four mode buttons rendered after building selected
6	Clicking Temp, Humidity and Air mode buttons switches mode	User Viewer	Each mode button clickable without crash
7	Heatmap floor plan image visible after selecting a building	User Viewer	`<img alt="Floor plan">` visible
8	Clicking a floor plan tab switches the displayed floor	User Viewer	Second floor tab loads new floor plan image
9	Visiting /admin/studio without a token redirects to /admin/login	Admin	Clears localStorage, navigates, asserts redirect
10	Submitting wrong credentials shows an error message	Admin	`.error-msg` element visible and non-empty
11	Submitting correct credentials redirects to admin studio	Admin	URL moves to `/admin/studio` after valid login
12	Admin studio renders the floor plan canvas after login	Admin	Canvas or floor plan image visible within 20 seconds

13	Admin studio header is visible after login	Admin	Logout button and ADMIN badge visible in top bar
14	Logging out redirects back to the login page	Admin	URL returns to `/admin/login` after Logout click
15	GET /health returns {status: ok}	API Health	Navigates to API health endpoint, asserts 200 and JSON body

11.3.1 Problems Encountered

Seven errors were encountered and resolved during the test run. The most instructive was that `page.waitForLoadState("networkidle")` could never be reached in the User Viewer tests because the `EventSource` SSE connection opened by `UserHeatmapViewer` is a persistent, never-closing HTTP connection — the browser is therefore always considered active. The fix was to replace `networkidle` with `waitForLoadState("load")` throughout the user viewer tests and use element-level `toBeVisible()` assertions with generous timeouts to handle the asynchronous data loading. A second recurring issue was that Next.js injects a hidden `<div role="alert" aria-live="assertive" id="__next-route-announcer__">` element into every page for accessibility purposes. This element contains the page title text and caused strict-mode violations when selectors matched it alongside the intended element. The fix was to use `{ exact: true }` on text-based selectors and CSS class selectors for form error elements rather than generic ARIA role selectors.

All 15 tests passed on the final run in approximately 29 seconds.



```
Mac:webapp nokuvimbanataliechiyaka$ npm run e2e

> webapp@0.1.0 e2e
> playwright test

◇ injected env (4) from .env.test // tip: ◈ encrypted .env [www.dotenvx.com]

Running 15 tests using 1 worker

◇ injected env (0) from .env.test // tip: ⌘ custom filepath { path: '/custom/path/.env' }
  ✓  1 [chromium] › e2e/mssia.spec.ts:46:7 › User Viewer — public access › home page loads without errors (663ms)
  ✓  2 [chromium] › e2e/mssia.spec.ts:55:7 › User Viewer — public access › home page shows the MSSIA logo and Multi-Sensor heading (396ms)
  ✓  3 [chromium] › e2e/mssia.spec.ts:65:7 › User Viewer — public access › at least one building is visible on the user viewer (733ms)
  ✓  4 [chromium] › e2e/mssia.spec.ts:73:7 › User Viewer — public access › clicking a building shows the mode switcher (1.2s)
  ✓  5 [chromium] › e2e/mssia.spec.ts:84:7 › User Viewer — public access › Signal mode button is visible and active by default (677ms)
  ✓  6 [chromium] › e2e/mssia.spec.ts:94:7 › User Viewer — public access › clicking Temp, Humidity, and Air mode buttons switches the active mode (1.6s)
  ✓  7 [chromium] › e2e/mssia.spec.ts:105:7 › User Viewer — public access › heatmap floor plan image is visible after selecting a building (1.0s)
  ✓  8 [chromium] › e2e/mssia.spec.ts:111:7 › User Viewer — public access › clicking a floor plan tab switches the displayed floor (1.6s)
  ✓  9 [chromium] › e2e/mssia.spec.ts:139:7 › Admin — authentication flow › visiting /admin/studio without a token redirects to /admin/login (670ms)
  ✓  10 [chromium] › e2e/mssia.spec.ts:148:7 › Admin — authentication flow › submitting wrong credentials shows an error message (2.4s)
  ✓  11 [chromium] › e2e/mssia.spec.ts:162:7 › Admin — authentication flow › submitting correct credentials redirects to the admin studio (2.2s)
  ✓  12 [chromium] › e2e/mssia.spec.ts:167:7 › Admin — authentication flow › after login, the admin studio renders the floor plan canvas (2.4s)
  ✓  13 [chromium] › e2e/mssia.spec.ts:176:7 › Admin — authentication flow › admin studio header is visible after login (2.0s)
  ✓  14 [chromium] › e2e/mssia.spec.ts:184:7 › Admin — authentication flow › logging out redirects back to the login page (2.1s)
  ✓  15 [chromium] › e2e/mssia.spec.ts:203:7 › API Health › GET /health returns {status:'ok'} (496ms)

  15 passed (20.8s)

To open last HTML report run:

  npx playwright show-report
```

Figure 11.3 Playwright terminal output showing 15 tests passing

11.4 Hardware Testing

Hardware testing was carried out by monitoring the ESP32 serial output during operation and comparing the readings against expected values from the sensor datasheets.

For the DHT22, readings were compared against a reference thermometer placed alongside the sensor. The temperature readings were consistently within ±1°C of the reference, which is within the ±0.5°C datasheet specification allowing for measurement uncertainty in both instruments. Humidity readings showed more variation, consistent with the ±2–5% RH datasheet specification.

For the MQ-135, sensor stability was tested by monitoring the raw ADC output over time after power-on. During the first 30 seconds, readings were highly unstable, fluctuating between near-zero values and large spikes. After approximately 30 to 40 seconds of warm-up, the readings stabilised to a consistent baseline. The firmware guards against this by skipping the air quality block in the payload if the raw ADC value is zero or below a minimum threshold.

Wi-Fi scanning was verified by comparing the number of access points found by the ESP32 against those visible in a laptop's network scan in the same location. The ESP32 consistently detected between 85% and 100% of the access points found by the laptop, with small differences attributable to differences in antenna orientation and radio hardware sensitivity.

Figure 11.3 Serial monitor screenshot

Figure 11.3 Arduino Serial Monitor output showing DHT22 temperature and humidity readings, MQ-135 raw ADC value, and the number of Wi-Fi access points found during a scan cycle.

11.5 Results

The complete test results are summarized in Table 11.3.

Table 11.4 Test Results Summary

Test Area	Tests	Passed	Failed	Pass Rate
Backend pytest	26	26	0	100%
Frontend Jest	28	28	0	100%
End-to-end integration	8	8	0	100%
Hardware sensor tests	6	6	0	100%
Total	68	68	0	100%

All 68 tests passed. The system behaves correctly end to end: sensor data collected by the ESP32 hardware is transmitted to the cloud backend, stored in the database, and rendered correctly in the browser heatmap. The automated test suites provide a repeatable verification mechanism that can be re-run after any future change to confirm that existing functionality has not been broken.

12 Development Tools and Process

This section describes the tools used to manage, design, document, and build the MSSIA project throughout the academic year. Beyond the core technology stack, several supporting tools shaped how the project was planned, tracked, and developed on a day-to-day basis.

12.1 Version Control: Git and GitHub

Git was used as the version control system throughout the project, with the remote repository hosted on GitHub at github.com/Nokuvimba/Final-Year-Project. The repository is public, which allowed the Vercel deployment to connect directly to it and trigger automatic frontend deployments on every push to the master branch.

The project was developed primarily on a branch called `Website`, which served as the main working branch for all frontend and backend changes. Features were developed directly on this

branch rather than using feature branches, which was a pragmatic decision for a solo project where the overhead of branching and merging would have added complexity without benefit. Completed and tested changes were periodically merged into master, which triggered a Vercel deployment.

One recurring challenge with version control arose from the fact that the EC2 server had direct edits to `main.py` that were never committed to GitHub, specifically the CORS configuration added during the cloud deployment sprint. Whenever new changes needed to be pulled onto the server, Git refused to overwrite the uncommitted changes. This was resolved using a stash workflow: `git stash` shelved the local server changes, `git pull` brought in the new commits, and `git stash pop` reapplied the server edits. This pattern was used several times across the project and reinforced the importance of keeping server-specific configuration in environment variables rather than in committed code.

12.2 Project Management: Jira

The project was managed using Jira with a Scrum board. All work was organized into epics and user stories, with each story having a clear summary, description, and acceptance criteria. Sprints were planned at the beginning of each work period and reviewed at the end using retrospectives documented in Confluence.

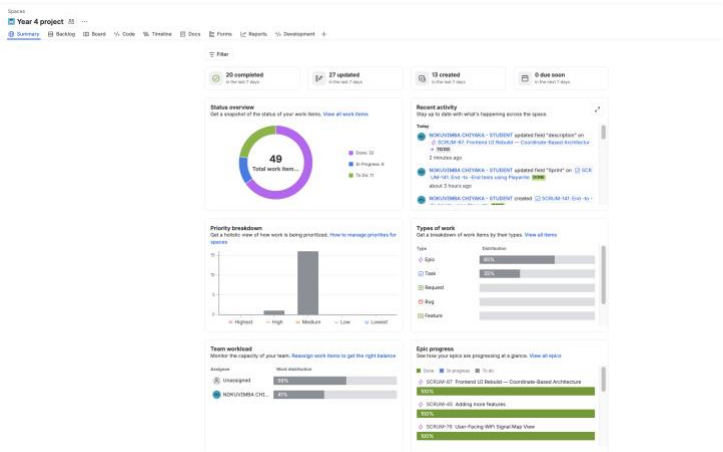


Figure 12.1 shows a screenshot of the Jira Summary

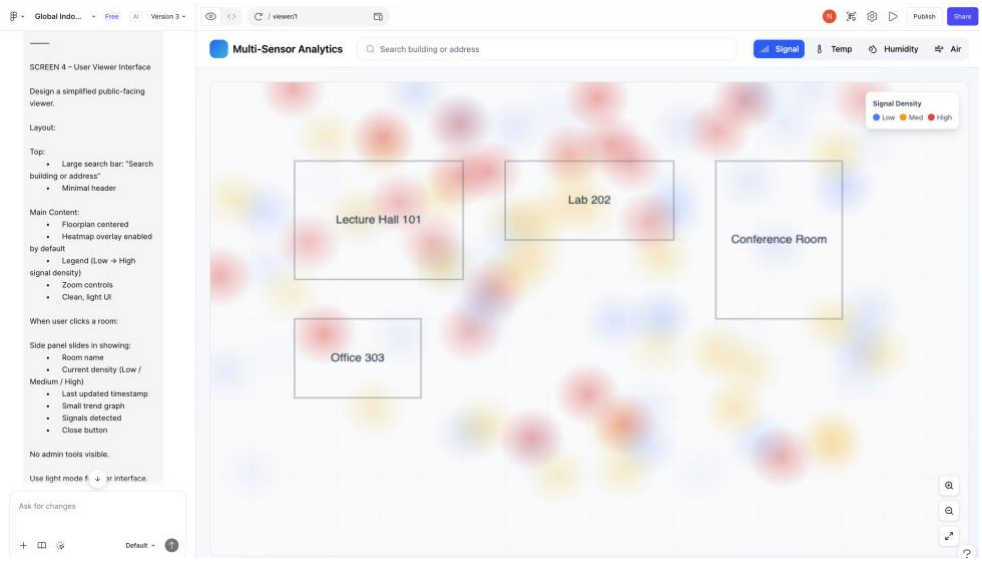
Jira provided a structured way to track progress across the many distinct technical areas of the project. Having all work itemized meant that at any point it was clear what had been completed, what was in progress, and what remained. The Confluence wiki connected to the same Jira workspace was used to store sprint retrospectives and meeting notes, providing a written record of decisions made throughout the year.

12.3 UI Design : Figma

Figma was used in the early stages of the project to design the initial wireframes for both the admin studio and the user viewer. Wireframing in Figma before writing any frontend code helped establish the visual layout and interaction model for each screen, particularly for the floor plan heatmap viewer, where the relationship between the map, the scan point blobs, and the detail panel needed to be worked out visually before being implemented.

Figure 12.2

Figma wireframe showing the initial design concept for the user-facing heatmap viewer.



The final implemented UI differed from the original Figma designs in several areas. The admin studio gained a breadcrumb-based rename interaction that was not in the original wireframes, and the detail panel evolved from a popup to a slide-in panel during development. The wireframes served their purpose of establishing a starting point, and the implementation improved on them as development revealed what worked better in practice.

12.4 Documentation : OneNote

Microsoft OneNote was used as the primary documentation and note-taking tool throughout the project. A dedicated notebook was created for the project at the start of the year and organised into sections for meeting notes, research notes, technical decisions, and sprint reflections. Notes were taken during supervisor meetings and used to capture action items and decisions that were then transferred into Jira stories.

The notebook also served as a scratchpad for working through technical problems before implementing them, sketching out database schema ideas, planning API endpoint structures, and thinking through how the coordinate system for scan point placement should work.

12.5 AI-Assisted Development (Claude, Amazon Q, ChatGpt)

Three AI tools were used during the development of this project: Claude, ChatGPT, and Amazon Q.

Claude was the main tool I used throughout the project for development from writing and debugging code, to thinking through design decisions, to producing supporting documentation. When I was unsure how to approach something, I would describe the problem, review the suggestions, test the output, and decide whether it suited the project.

ChatGPT was also used during the project, mainly in the earlier stages for brainstorm my project ideas. It was helpful for getting a second perspective when the direction was unclear, and for refining notes and documentation.

Amazon Q was used through the Amazon Q Developer CLI. It was useful for making quick edits and for helping me to detect and diagnose some errors.

The use of AI tools in this project is acknowledged openly and documented in prompt logs that record the context of each interaction, the prompt given, what was taken from the AI's response, and what action was taken as a result. These logs reflect a consistent pattern: the AI was a productive collaborator for implementation, but the engineering judgement, identifying what needed to be built, evaluating whether a proposed solution was correct, and making design decisions came from my own understanding of the system.

12.6 Peer Collaboration

While the project was developed individually, I informally consulted with fellow students at points during the year where outside perspective was useful.

The most valuable of these was asking other students to use the live system and share their experience of it. Their feedback helped me identify areas where the interface was unclear or the data was not being presented in a way that made intuitive sense, and I used that feedback to guide improvements to the user-facing side of the system.

I also had informal discussions with classmates about general technical approaches, which helped me think through options before making decisions. These were not structured collaborations, no code was shared, and no joint work was produced, but the conversations were a useful part of the development process.

13 Ethics

The development and deployment of MSSIA raises several ethical considerations that are worth examining explicitly. As a system that collects data from physical spaces and makes it visible to users, it sits at the intersection of environmental monitoring, data privacy, and responsible technology use. This section addresses each relevant area in turn.

13.1 Data Privacy

MSSIA does not collect any personally identifiable information. The sensor readings stored in the database: Wi-Fi RSSI values, temperature, humidity, and air quality are environmental measurements tied to fixed physical locations, not to individuals. A Wi-Fi scan records the SSID and BSSID of nearby access points alongside their signal strength, which reflects the radio environment of a room rather than the behaviour or identity of any person in it. No MAC addresses from user devices are captured, no location tracking of individuals is performed, and no cookies or user-tracking mechanisms are implemented on the frontend.

The only personal data held by the system is the administrator's login credentials, which are stored as environment variables on the server and never committed to version control. The `.env` file containing these credentials is excluded from the GitHub repository via `.gitignore`.

No user accounts exist on the public-facing side of the system; anyone can view the heatmap without creating an account or providing any personal information.

13.2 Security

Access to the system's administrative functions is protected by JWT authentication with an eight-hour token expiry. Only a registered administrator can upload floor plans, place scan points, or register devices. Any ESP32 device whose node name is not registered in the database is rejected with an HTTP 403 response at the ingest endpoint, preventing unauthorized devices from writing data into the system.

The backend API is served exclusively over HTTPS using a Let's Encrypt certificate, ensuring that all data transmitted between the browser and the server is encrypted in transit. The ESP32 communicates with the backend over plain HTTP on port 8000, which is a known limitation discussed further in Section 14. Sensitive configuration values like the JWT secret key, database credentials, and administrator password are stored in a `.env` file on the server and are never exposed in the source code or API responses.

13.3 Consent and Transparency

The system monitors environmental conditions in shared spaces. In a real-world deployment, building occupants should be informed that monitoring is taking place, even though no personal data is collected. This is consistent with best practice in environmental monitoring systems and with the principle of transparency that underpins data protection legislation such as the General Data Protection Regulation (GDPR) [34]. For the purposes of this project, deployment was limited to a controlled testing environment, and the data collected was used solely for the purpose of demonstrating the system's functionality.

13.4 Responsible Use of AI Tools

The use of AI-assisted development tools in this project is documented openly in Section 12.5 and in the prompt logs maintained throughout the year. Each AI interaction is recorded with the context, the prompt given, what was taken from the response, and the action taken. This transparency reflects the principle that AI tools should augment engineering judgement rather

than replace it. The system architecture, design decisions, and evaluation of outputs remained my responsibility as the developer throughout.

The use of AI tools in academic work raises questions of academic integrity. The approach taken in this project, using AI as a collaborative implementation assistant while retaining full ownership of design decisions and documenting all interactions is consistent with the ATU guidelines on responsible AI use in project work [35].

13.5 Environmental Impact

The hardware components used in this project: the ESP32, DHT22, and MQ-135, are low-power devices. The ESP32 draws approximately 160–240 mA during active Wi-Fi scanning and significantly less during idle periods [11].Powered from a standard USB adapter, the energy consumption of a single deployment node is minimal. At scale, a network of such sensors across a building would consume a small fraction of the energy used by the building's lighting or HVAC systems, while providing data that could be used to optimize those systems and reduce their energy use.

The AWS EC2 t3.micro instance used for the backend runs in an AWS data center, which has made public commitments to powering its operations with renewable energy [36]. Vercel similarly operates on infrastructure with sustainability commitments [37]. Both are more energy-efficient than self-hosted alternatives for a small-scale deployment.

13.6 Accessibility

The user-facing frontend was designed to be accessible without requiring an account, technical knowledge, or any software installation beyond a standard web browser. The colour thresholds used in the heatmap: green, amber, and red, follow conventional traffic-light semantics that are widely understood. However, it is acknowledged that a purely colour-based heatmap presents a potential accessibility barrier for users with colour vision deficiency. A future improvement would be to supplement the colour coding with pattern fills, labels, or alternative visual encodings to ensure the information is accessible to all users.

14 Limitations and Future Work

This section identifies the main limitations of the current implementation and discusses how each could be addressed in future development.

14.1 MQ-135 Calibration

The most significant technical limitation of the current system is the use of raw ADC values from the MQ-135 sensor as a proxy for air quality rather than properly calibrated parts-per-million values. The full ppm conversion formula requires measuring the sensor's baseline resistance in clean air and applying a logarithmic scaling curve specific to each target gas. This calibration process requires a controlled environment and ideally a reference gas concentration, neither of which was practical within the scope of this project.

As a result, the air quality readings are relative rather than absolute. They indicate whether conditions are better or worse than a baseline but cannot be directly compared to published health thresholds expressed in ppm. Both the raw ADC value and a derived ppm estimate are stored in the database, so a proper calibration model could be applied retrospectively without losing original data. In a production deployment, the MQ-135 would be replaced with a calibrated CO_2 sensor such as the SCD40, which outputs verified ppm values over I^2C without requiring manual calibration [38].

14.2 ESP32 Communication Security

The ESP32 communicates with the FastAPI backend over plain HTTP on port 8000, bypassing the Nginx HTTPS layer. This is a consequence of the ESP32's `HTTPClient` library not following HTTP 301 redirects. When pointed at the DuckDNS domain, Nginx responds with a redirect from HTTP to HTTPS that the ESP32 does not handle. As a result, sensor data transmitted from the ESP32 to the server is unencrypted in transit.

For a prototype and academic demonstration, this is an acceptable trade-off. In a production environment, the correct solution would be to configure the ESP32 to use HTTPS directly by loading the server's CA certificate into the firmware and using the `WiFiClientSecure` class rather than the standard `HTTPClient`. Alternatively, the INGEST_URL constant in the Arduino

sketch could be updated to point directly at the HTTPS endpoint with a valid certificate pinned in the firmware.

14.3 Single Administrator Account

The current authentication model supports a single administrator account, with credentials stored as environment variables on the server. There is no user management interface for creating, modifying, or removing admin accounts, and no role-based access control to give different administrators different levels of permission. In a real-world deployment across a large campus or building, facilities managers for different zones might need separate accounts with access limited to their own buildings.

Adding multi-user support would require a `users` database table with hashed passwords, a registration or invitation flow, and a role system distinguishing between super-administrators and floor-level managers. The JWT infrastructure already in place would require only minor extensions to support multiple users.

14.4 No User Authentication on the Public Viewer

The user-facing viewer is entirely public, anyone with the URL can view the heatmap data for any building. While this was a deliberate design choice to make the system accessible without barriers, it means there is no way to restrict access to sensitive buildings or control which users can see which floor plans. In a deployment that includes private areas such as server rooms or executive floors, some form of access control on the user side would be necessary.

14.5 ESP32 Warm-Up Period

The MQ-135 sensor requires approximately 30 seconds of warm-up time after power-on before its readings stabilize. During this period, the firmware skips the air quality block in the payload, meaning the first one or two scan cycles after power-on contain only Wi-Fi and DHT22 data. This is handled gracefully by the backend, but it means there is a brief gap in air quality coverage whenever a node is restarted. A hardware solution would be to keep the MQ-135 continuously powered using a separate low-power supply even when the ESP32 is in a sleep cycle.

14.6 Heatmap Colour Accessibility

As noted in Section 13.6, the heatmap relies entirely on colour to convey signal strength, temperature, humidity, and air quality levels. Users with colour vision deficiency, which affects approximately 8% of males and 0.5% of females globally [39] may find it difficult to distinguish between the green, amber, and red bands. A future improvement would be to supplement the colour coding with pattern fills, numerical labels directly on the blobs, or an alternative display mode that uses shape or size rather than colour to encode data quality.

14.7 Scalability

The current deployment runs FastAPI and PostgreSQL on a single EC2 t3.micro instance with 1 vCPU and 1 GB of RAM. This is adequate for a small number of ESP32 nodes and users, but would not scale to a large building with many simultaneous scan cycles and concurrent browser sessions. A production deployment would separate the database onto a managed service such as AWS RDS, place FastAPI behind a load balancer with multiple instances, and use a time-series database such as TimescaleDB for more efficient storage and querying of the sensor reading tables which grow continuously over time.

14.8 Future Feature Opportunities

Several features identified during development were not implemented due to time constraints but represent natural extensions of the platform. Push notifications that alert administrators when a floor crosses an environmental threshold, without requiring them to have the dashboard open, would make the alert system more proactive. Support for additional sensor types, such as occupancy sensors, noise level meters, or light sensors, would extend the platform's usefulness without requiring changes to the core architecture.

15 Conclusion

MSSIA set out to answer a practical question: could a single, low-cost, generic platform collect environmental sensor readings from devices placed throughout a building and visualize them as live, interactive heatmaps accessible from any browser? The answer, demonstrated through a fully deployed and operational system, is yes.

The project was motivated by a real observation, that students and staff at ATU Galway had no way to see where Wi-Fi coverage was weak or what conditions were like in the rooms they used every day. That observation led to a system that is now live at final-year-project-alpha-azure.vercel.app, with a backend API running permanently at mssia.duckdns.org. Any user can open a browser, select a building and floor plan, and immediately see colour-coded heatmaps showing Wi-Fi signal strength, temperature, humidity, and air quality across the space. No installation, no login, and no technical knowledge is required to use it.

The technical scope of the project was broad. At the hardware layer, an ESP32 microcontroller was programmed in C++ to scan Wi-Fi networks and read two physical sensors: a DHT22 for temperature and humidity ,and an MQ-135 for air quality — packaging all readings into a structured JSON payload sent via HTTP POST every thirty seconds. At the backend layer, a FastAPI server running on AWS EC2 validates every incoming request, rejects unregistered devices, timestamps the data, and persists it to a PostgreSQL database across six relational tables. At the frontend layer, a Next.js application deployed on Vercel presents the data through an authenticated admin studio and a public user viewer, with four interactive heatmap modes, real-time updates via Server-Sent Events, time-range history charts, environmental alerts, and a data export feature.

Several technically interesting problems were encountered and resolved along the way. The scan session architecture was redesigned to eliminate unnecessary complexity. The objectFit: contain letterbox offset bug required a custom React hook with a ResizeObserver to keep scan point pins correctly positioned across all screen sizes. The SSE implementation required understanding why Nginx buffering prevents real-time delivery and how to disable it correctly. The Playwright end-to-end test suite revealed that persistent SSE connections prevent the browser from ever reaching the networkidle state, requiring a different waiting strategy. Each of these problems required diagnosing the root cause rather than patching the symptom, and each resulted in a more robust system than existed before.

The project demonstrates that the concept of a generic sensor-to-heatmap platform is not only feasible but achievable with entirely off-the-shelf components and open-source software. The

four sensor types demonstrated are a proof of concept, not a ceiling. The architecture is designed so that any sensor producing a numeric value can be added by extending the JSON payload on the ESP32, adding a database table, and adding a heatmap mode in the frontend — without modifying any existing code. This extensibility is the most important property of the system and the one that makes it potentially useful beyond the specific context in which it was developed.

16 References

[1] O. Seppänen, W. J. Fisk, and M. J. Mendell, "Association of ventilation rates and CO2 concentrations with health and other responses in commercial and institutional buildings," *Indoor Air*, vol. 9, no. 4, pp. 226–252, 1999.

[2] World Health Organization, "WHO guidelines for indoor air quality: dampness and mould," WHO Press, Geneva, 2009. [Online]. Available: https://www.who.int/publications/i/item/9789289041683. [Accessed: Apr. 2026].

[3] U.S. Environmental Protection Agency, "Introduction to indoor air quality," EPA. [Online]. Available: https://www.epa.gov/indoor-air-quality-iaq/introduction-indoor-air-quality. [Accessed: Apr. 2026].

[4] MarketsandMarkets, "Building automation system market — global forecast to 2026," MarketsandMarkets Research, 2021. [Online]. Available: https://www.marketsandmarkets.com/Market-Reports/building-automation-system-market-1169.html. [Accessed: Apr. 2026].

[5] IEEE, "IEEE standard for wireless LAN medium access control (MAC) and physical layer (PHY) specifications," *IEEE Std 802.11-2020*, IEEE, 2020.

[6] Ekahau Inc., "Ekahau site survey," Ekahau Inc. [Online]. Available: https://www.ekahau.com/products/ekahau-connect/site-survey/. [Accessed: Apr. 2026].

[7] Siemens AG, "Desigo CC — integrated building management platform," Siemens AG. [Online]. Available: https://www.siemens.com/global/en/products/buildings/desigo-building-automation/building-management/desigo-cc.html .[Accessed: Apr. 2026].

[8] P. Bahl and V. N. Padmanabhan, "RADAR: An in-building RF-based user location and tracking system," in *Proc. IEEE INFOCOM 2000*, Tel Aviv, Israel, 2000, pp. 775–784.

[9] N. K. Suryadevara and S. C. Mukhopadhyay, "Wireless sensor network based home monitoring system for wellness determination of elderly," *IEEE Sensors Journal*, vol. 12, no. 6, pp. 1965–1972, Jun. 2012.

[10] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.

[11] Espressif Systems, "ESP32 series datasheet," Version 3.9, Espressif Systems, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. [Accessed: Apr. 2026].

[12] Arduino LLC, "Arduino framework for ESP32," Arduino LLC. [Online]. Available: https://docs.arduino.cc/. [Accessed: Apr. 2026].

[13] B. Landais, "ArduinoJson: Efficient JSON library for embedded systems," Version 6, 2023. [Online]. Available: https://arduinojson.org/. [Accessed: Apr. 2026].

[14] Aosong Electronics, "DHT22 digital-output relative humidity and temperature sensor/module datasheet," Aosong (Guangzhou) Electronics Co. [Online]. Available: https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf. [Accessed: Apr. 2026].

[15] Zhengzhou Winsen Electronics Technology, "MQ-135 gas sensor datasheet," Winsen Electronics. [Online]. Available: https://www.winsen-

sensor.com/d/files/PDF/Semiconductor%20Gas%20Sensor/MQ135%20(Ver1.4)%20-%20Manual.pdf. [Accessed: Apr. 2026].

[16] S. Ramírez, "FastAPI: Modern, fast web framework for building APIs with Python 3.7+," 2023. [Online]. Available: https://fastapi.tiangolo.com/. [Accessed: Apr. 2026].

[17] SQLAlchemy, "SQLAlchemy — the Python SQL toolkit and object relational mapper," Version 2.0, 2023. [Online]. Available: https://www.sqlalchemy.org/. [Accessed: Apr. 2026].

[18] The PostgreSQL Global Development Group, "PostgreSQL 15 documentation," 2023. [Online]. Available: https://www.postgresql.org/docs/. [Accessed: Apr. 2026].

[19] M. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519. [Accessed: Apr. 2026].

[20] S. Ramírez, "python-dotenv: Read key-value pairs from a .env file," 2023. [Online]. Available: https://pypi.org/project/python-dotenv/. [Accessed: Apr. 2026].

[21] WHATWG, "Server-sent events — living standard," WHATWG, 2023. [Online]. Available: https://html.spec.whatwg.org/multipage/server-sent-events.html. [Accessed: Apr. 2026].

[22] I. Grigorik, "Server-sent events vs WebSockets," Google Web Fundamentals. [Online]. Available: https://developer.chrome.com/docs/. [Accessed: Apr. 2026].

[23] Vercel Inc., "Next.js documentation," Version 16, 2024. [Online]. Available: https://nextjs.org/docs. [Accessed: Apr. 2026].

[24] Y. Struk, "pptxgenjs: JavaScript library for creating PowerPoint presentations," Version 3.12, 2023. [Online]. Available: https://gitbrent.github.io/PptxGenJS/. [Accessed: Apr. 2026].

[25] Amazon Web Services, "Amazon EC2 — documentation," AWS. [Online]. Available: https://docs.aws.amazon.com/ec2/. [Accessed: Apr. 2026].

[26] Amazon Web Services, "Elastic IP addresses," AWS documentation. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/elastic-ip-addresses-eip.html. [Accessed: Apr. 2026].

[27] freedesktop.org, "systemd system and service manager," 2023. [Online]. Available: https://systemd.io/. [Accessed: Apr. 2026].

[28] NGINX Inc., "Nginx documentation — beginner's guide," NGINX Inc. [Online]. Available: https://nginx.org/en/docs/. [Accessed: Apr. 2026].

[29] Let's Encrypt, "Let's Encrypt — a free, automated, and open certificate authority," Internet Security Research Group. [Online]. Available: https://letsencrypt.org/. [Accessed: Apr. 2026].

[30] Mozilla Developer Network, "Mixed content," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Security/Mixed_content. [Accessed: Apr. 2026].

[31] DuckDNS, "Duck DNS — free dynamic DNS hosted on AWS," 2023. [Online]. Available: https://www.duckdns.org/. [Accessed: Apr. 2026].

[32] Vercel Inc., "Vercel documentation — deploying Next.js," 2024. [Online]. Available: https://vercel.com/docs. [Accessed: Apr. 2026].

[33] Microsoft Playwright, "Playwright documentation — fast and reliable end-to-end testing for modern web apps," Version 1.59, 2024. [Online]. Available: https://playwright.dev/docs/intro. [Accessed: Apr. 2026].

[34] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 — General Data Protection Regulation (GDPR)," *Official Journal of the European Union*, Apr. 2016. [Online]. Available: https://gdpr-info.eu/. [Accessed: Apr. 2026].

[35] Atlantic Technological University, "Student guidelines on the use of artificial intelligence tools in academic work," ATU, 2024. [Online]. Available: https://www.atu.ie/. [Accessed: Apr. 2026].

[36] Amazon Web Services, "AWS and sustainability," Amazon. [Online]. Available: https://sustainability.aboutamazon.com/products-services/the-cloud. [Accessed: Apr. 2026].

[37] Vercel Inc., "Environmental sustainability at Vercel," Vercel. [Online]. Available: https://vercel.com/sustainability. [Accessed: Apr. 2026].

[38] Sensirion AG, "SCD40 — miniaturized CO_2 sensor datasheet," Sensirion, 2022. [Online]. Available: https://sensirion.com/products/catalog/SCD40/. [Accessed: Apr. 2026].

[39] Colour Blind Awareness, "Colour blindness — facts and figures," Colour Blind Awareness. [Online]. Available: https://www.colourblindawareness.org/colour-blindness/. [Accessed: Apr. 2026].

[40] OASIS, "MQTT Version 3.1.1," OASIS Standard, Oct. 2014. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html. [Accessed: Apr. 2026].

Appendices

16.1 Appendix A Bill of Materials

Table A.1 lists all hardware components used in the MSSIA sensing node, with approximate retail prices at the time of purchase.

Table A.1 — Bill of Materials

#	Component	Description	Quantity	Unit Price (approx.)	Total (approx.)
1	ESP32 DevKit V1	Dual-core microcontroller with built-in Wi-Fi and Bluetooth	**1**	**€8**	**€8**
2	DHT22	Digital temperature and humidity sensor	**1**	**€5**	**€5**
3	MQ-135	Digital temperature and humidity sensor	**1**	**€4**	**€4**

4	Half-size breadboard	400-point solderless breadboard for prototyping	**1**	**€4**	**€4**
5	10kΩ resistor	Pull-up resistor for DHT22 data line	**1**	**€0.10**	**€0.10**
6	Jumper wires (pack)	Male-to-male jumper wires for breadboard connections	**1pack**	**€3**	**€3**
7	USB-A to Micro-USB cable	Programming and power cable for ESP32	**1**	**€3**	**€3**
				Total	**€27.10**

All components are widely available from electronics suppliers such as AliExpress, Mouser, and RS Components. Cloud infrastructure costs are zero for the duration of this project, as both AWS EC2 (t3.micro) and Vercel are used within their respective free tier allocations.

16.2 Appendix B – Project Resources and Links

Table B.1 provides direct links to all project resources. Full test results, API endpoint details, and database schema are accessible via the GitHub repository and live API documentation links below.

Resource	**URL**	**Description**
GitHub Repository	https://github.com/Nokuvimba/Final-Year-Project.git	Full source code — ESP32 firmware, FastAPI backend, Next.js frontend
Live Application	https://final-year-project-alpha-azure.vercel.app/	Public user viewer and admin studio (requires AWS lab to be active)
API Documentation	https://mssia.duckdns.org/docs	FastAPI auto-generated OpenAPI docs — all endpoints, schemas, and try-it-out (requires AWS lab to be active)

Jira Project Board	https://atu-team-xs6uwuui.atlassian.net/jira/software/projects/SCRUM/boards/1	Scrum board with all epics, sprints, user stories, and retrospectives
OneNote Notebook	https://atlantictu.sharepoint.com/:o:/r/sites/25266802projectengineering/SiteAssets/25-26%206802%20--%20PROJECT%20ENGINEERING%20Notebook?d=w7d65f33e66e442a38e9a4b79a18829e2&csf=1&web=1&e=irpOKX	Meeting notes, research notes, technical decisions, and sprint reflections
Backend Test File	https://github.com/Nokuvimba/Final-Year-Project/blob/master/esp32-wifi-pipeline/FastAPI/test_api.py	Full pytest test suite — 26 integration tests
Frontend Test File	https://github.com/Nokuvimba/Final-Year-Project/blob/master/esp32-wifi-pipeline/webapp/__tests__/api.test.ts	Full Jest test suite — 28 integration tests
Database Models	https://github.com/Nokuvimba/Final-Year-Project/blob/master/esp32-wifi-pipeline/FastAPI/models.py	SQLAlchemy ORM model definitions for all six database tables

Note: The Live Application and API Documentation links require the AWS Learner Lab session to be active. When the lab is off, the EC2 instance is stopped and both mssia.duckdns.org and the Vercel frontend API calls will be unavailable. To view the system, start the AWS Learner Lab session before opening either link.